

INTERNAL MEMORY의 이해

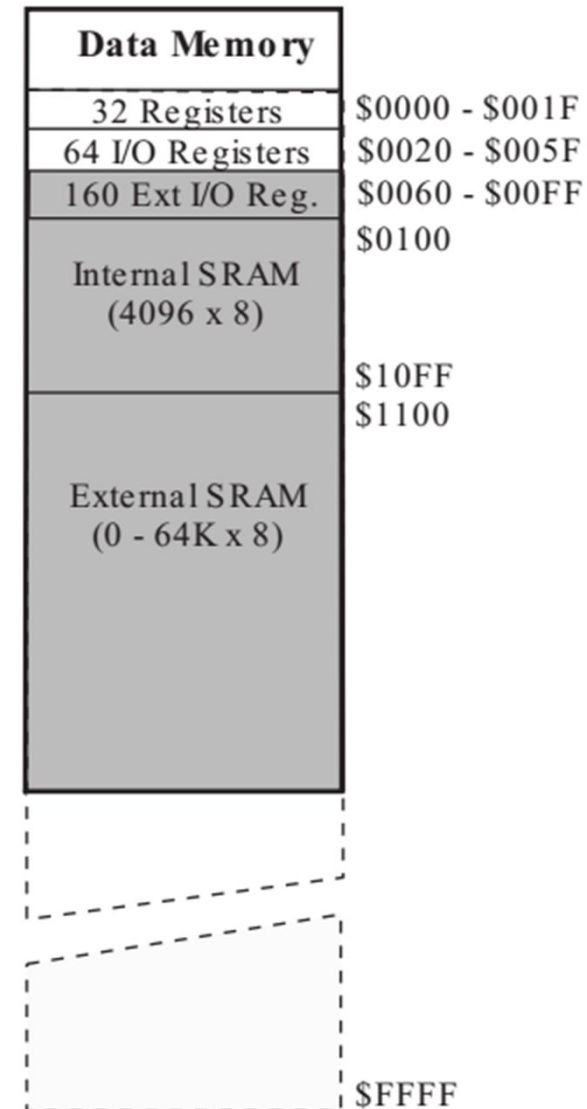
- ATmega128A의 데이터 메모리 구조
- Atmel Studio 디버깅 모드
- 내부 메모리와 레지스터 이해



엣지아이랩

ATMega128A의 데이터 메모리 구조

- ATMega128 내장 데이터 메모리
 - RAM
 - 레지스터
 - 범용레지스터
 - I/O레지스터
 - 확장 I/O 레지스터
 - SRAM
 - ROM
 - EEPROM



ATMega128A의 데이터 메모리 구조

- I/O 레지스터와 확장 I/O 레지스터
 - 내장된 각종 I/O 장치를 제어하기 위한 레지스터

주소	레지스터명	주소	레지스터명
\$00(\$20)	PINF	\$01(\$21)	PINE
\$02(\$22)	DDRE	\$03(\$23)	PORTE
\$10(\$30)	PIND	\$11(\$31)	DDRD
\$12(\$32)	PORTD	\$13(\$33)	PINC
\$14(\$34)	DDRC	\$15(\$35)	PORTC
\$16(\$36)	PINB	\$17(\$37)	DDRB
\$18(\$38)	PORTB	\$19(\$39)	PINA
\$1A(\$3A)	DDRA	\$1B(\$3B)	PORTA
\$34(\$54)	MCUCSR	\$35(\$55)	MCUCR
\$3F(\$3F)	SREG		

※ 위 표의 주소는 실제 IO 주소공간에서의 주소이며,
괄호안의 주소는 메모리 주소 공간에서의 주소(0x0020~\$0x005F)이다.

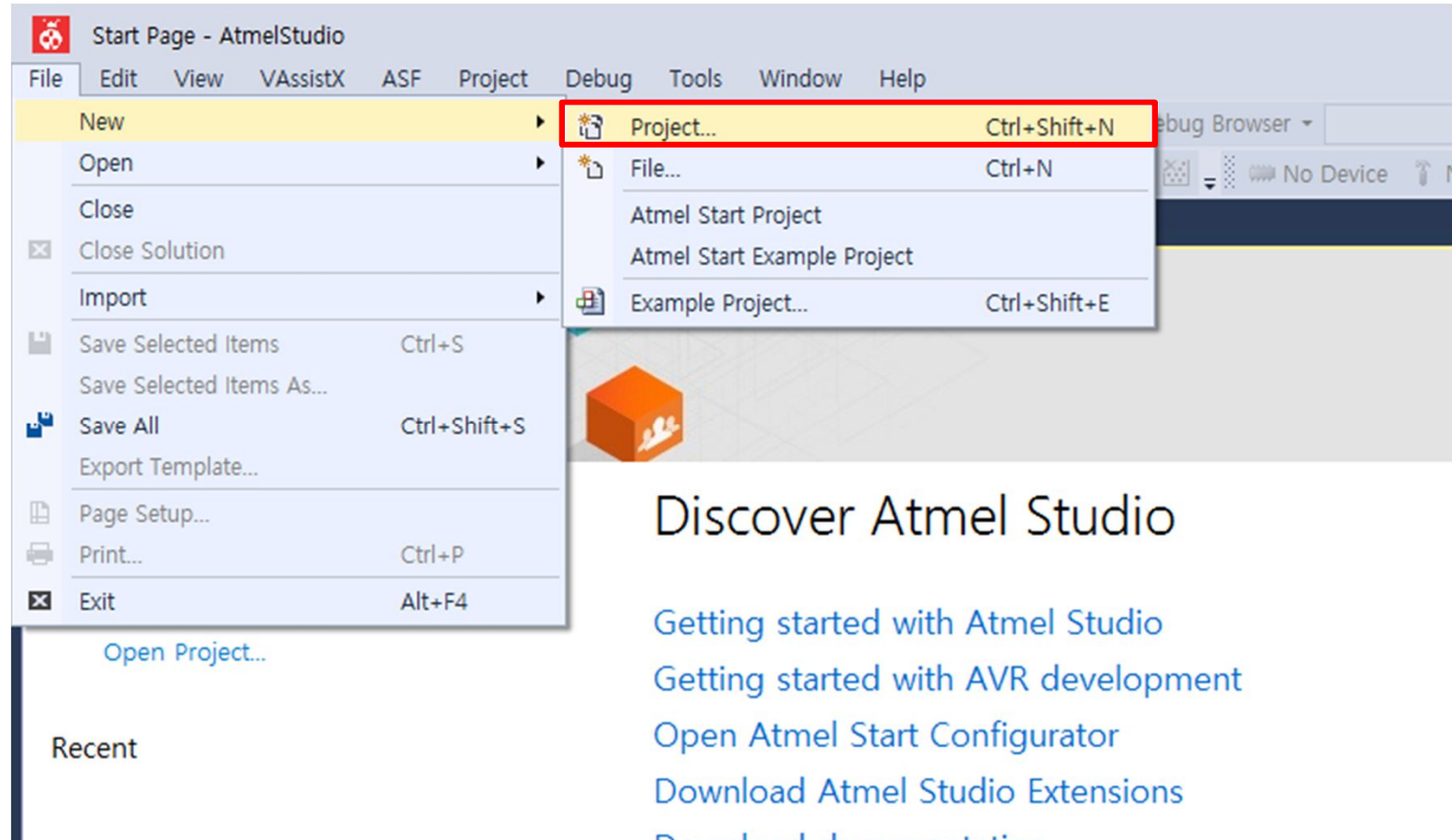
ATMega128A의 데이터 메모리 구조

- 확장 I/O 레지스터
 - ATMega128A에 추가된 각종 I/O 장치를 제어하기 위한 레지스터

주소	레지스터명	주소	레지스터명
\$61	DDRF	\$62	PORTF
\$63	PING	\$64	DDRG
\$65	PORTG	\$6A	EICRA
\$7C	ETIFR	\$7D	ETIMSK

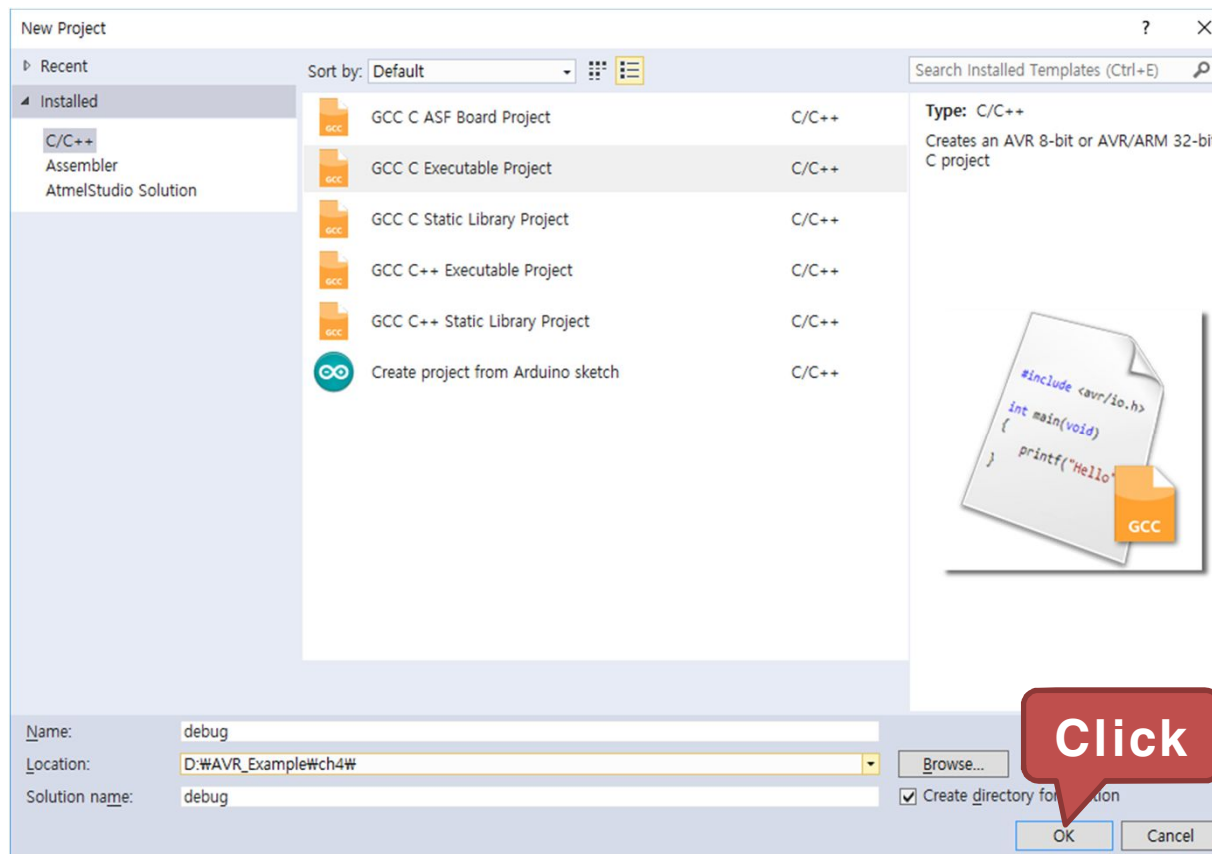
Atmel Studio 7.0 디버깅 모드

- File → New → Project... 을 클릭하여 새 프로젝트를 생성



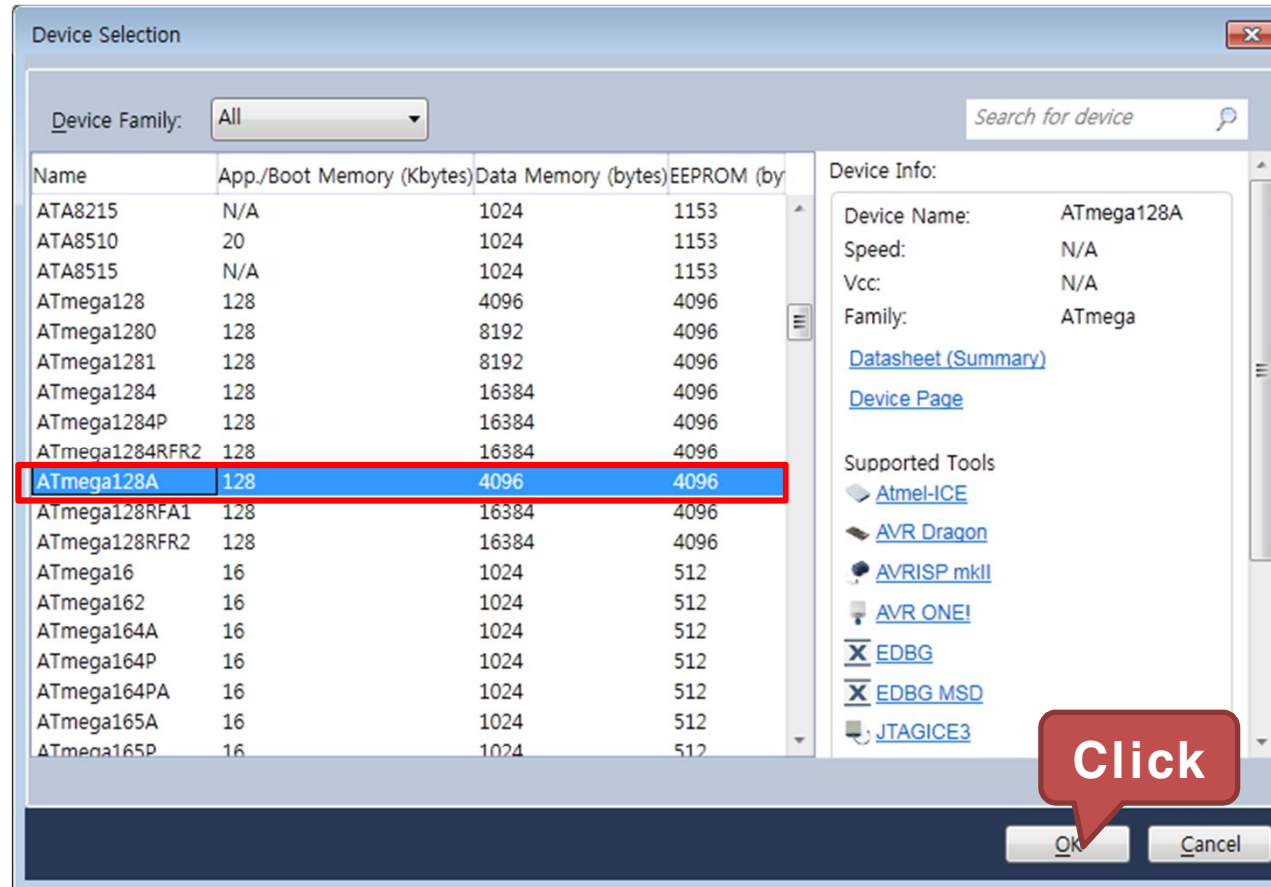
Atmel Studio 7.0 디버깅 모드

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 04_DEBUG_Example
 - Location : D:\WAVR_Example



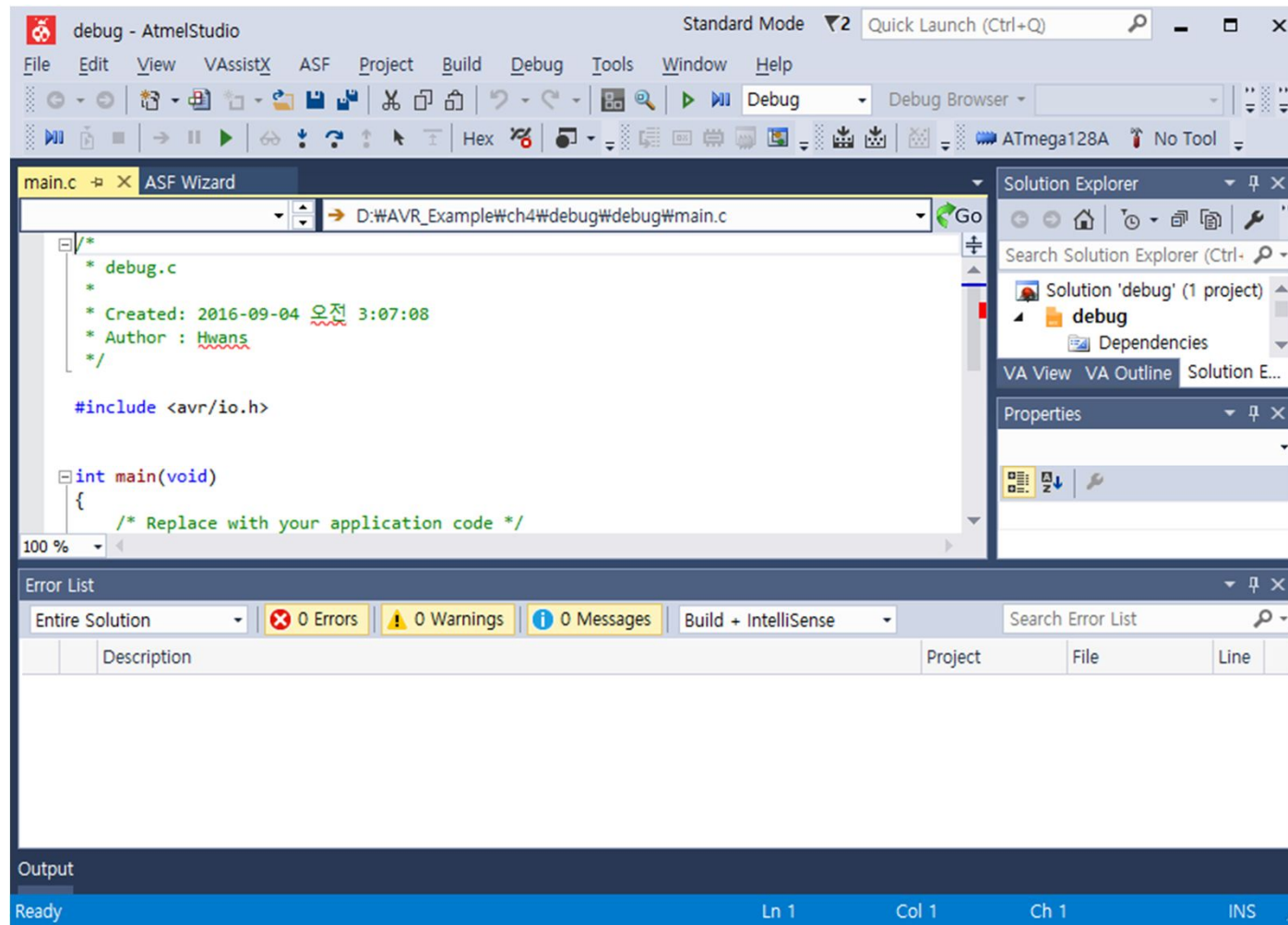
Atmel Studio 7.0 디버깅 모드

- ATmega128A를 선택하고 OK를 클릭



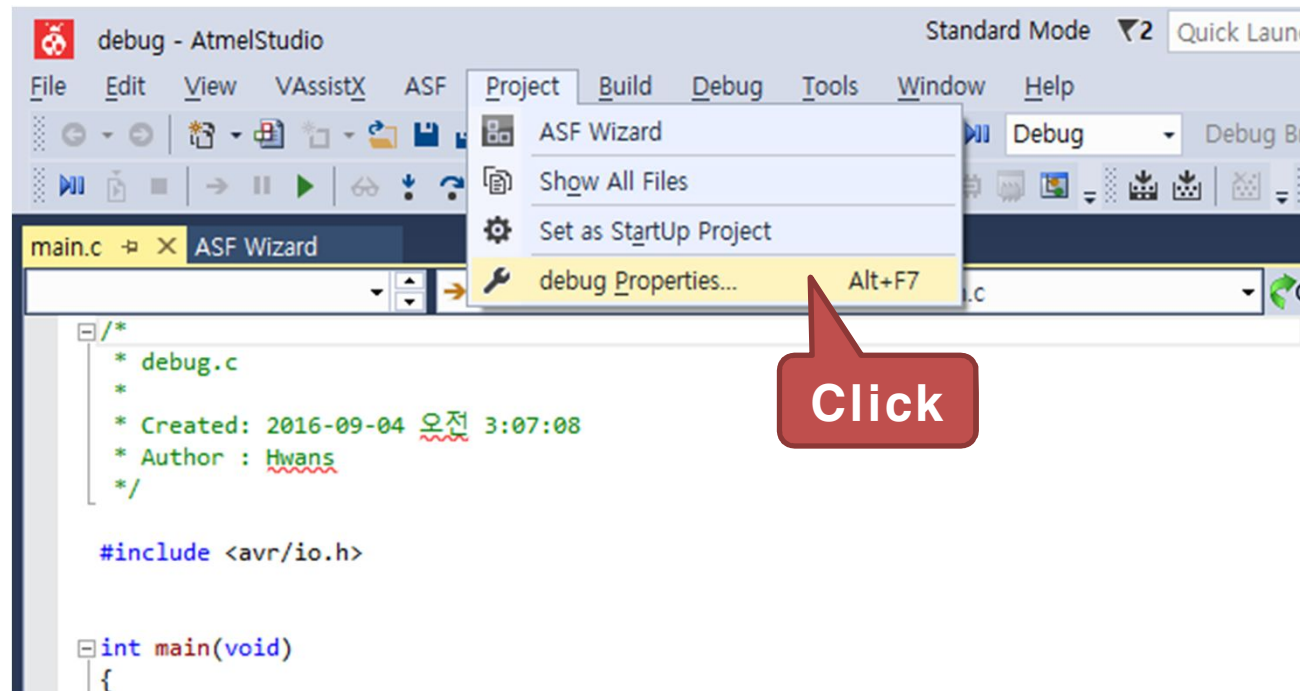
Atmel Studio 7.0 디버깅 모드

- Debug 프로젝트 생성 완료



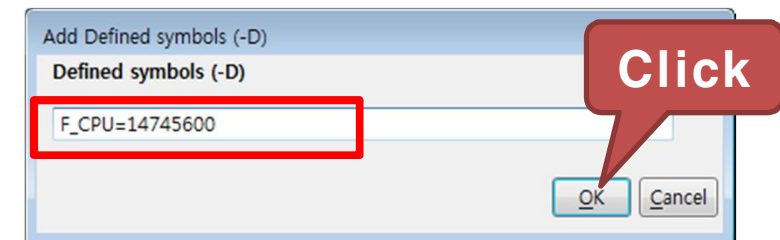
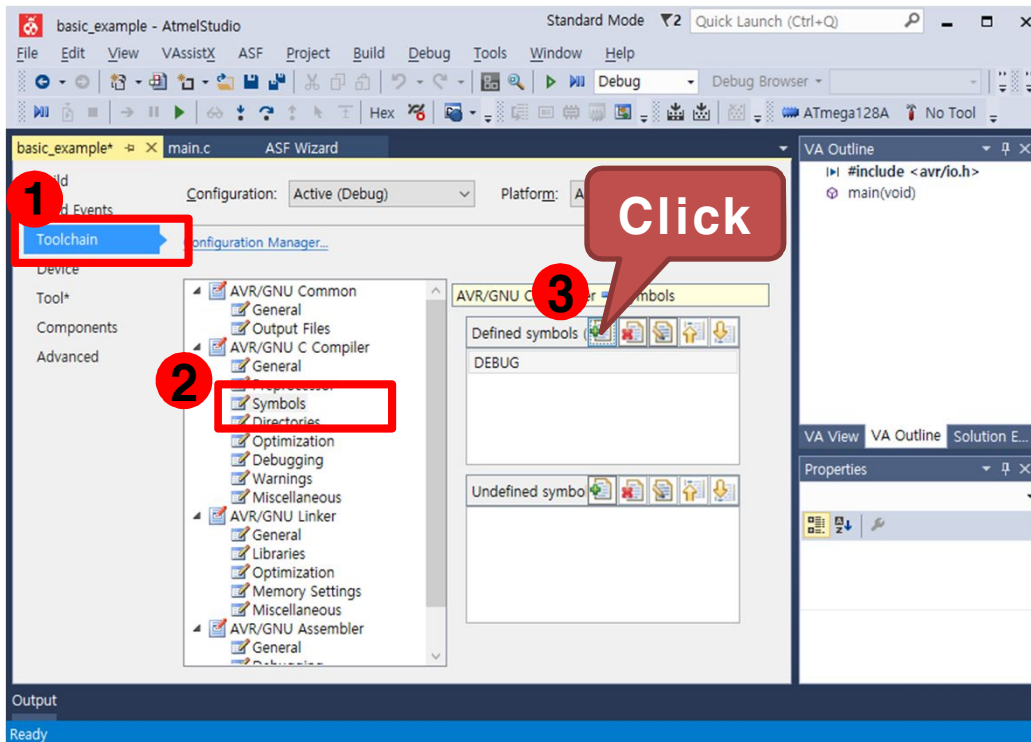
Atmel Studio 7.0 디버깅 모드

- 프로젝트 설정
 - Project 탭에서 "04_DEBUG_Example Properties..." 를 선택



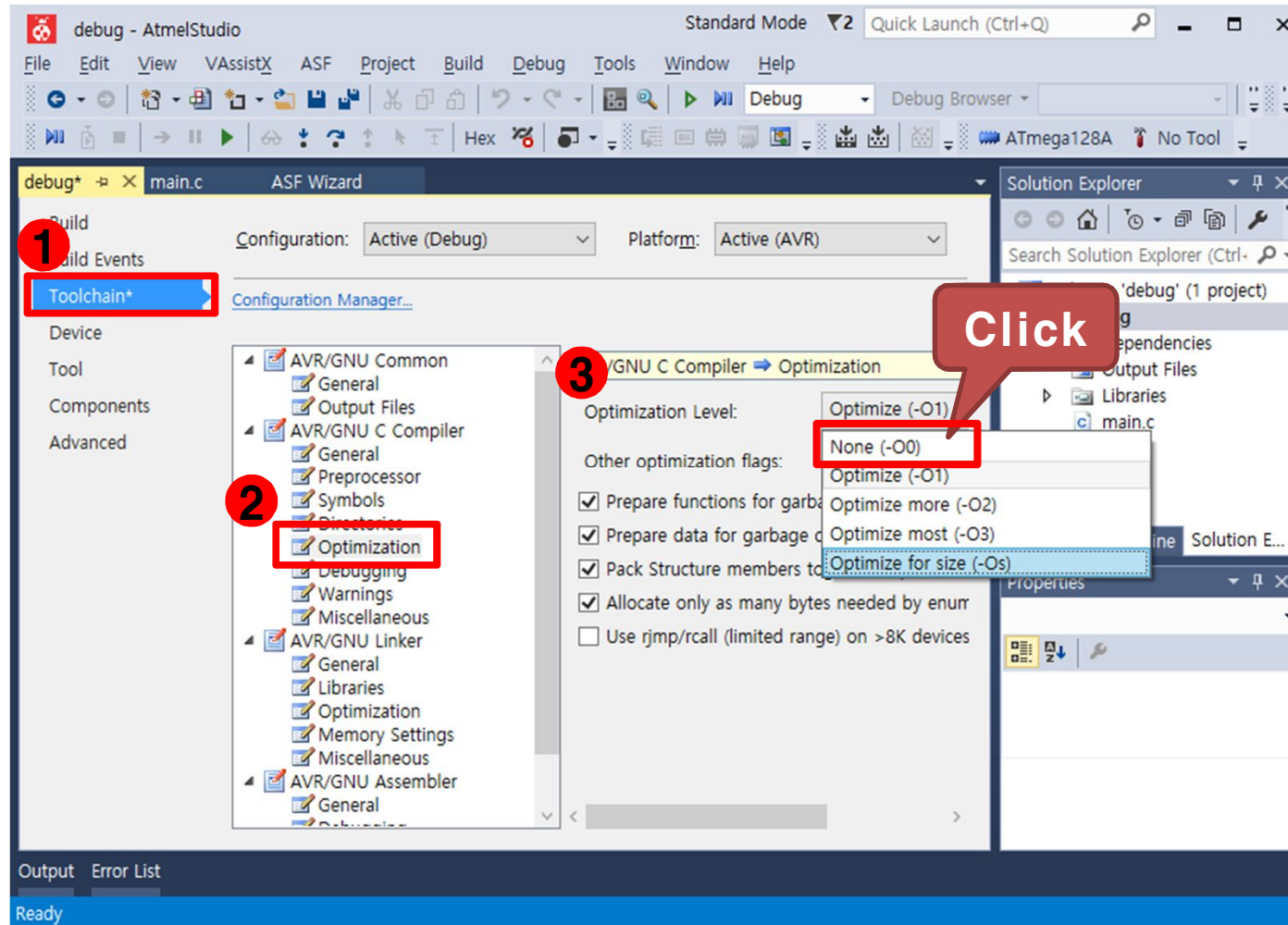
Atmel Studio 7.0 디버깅 모드

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



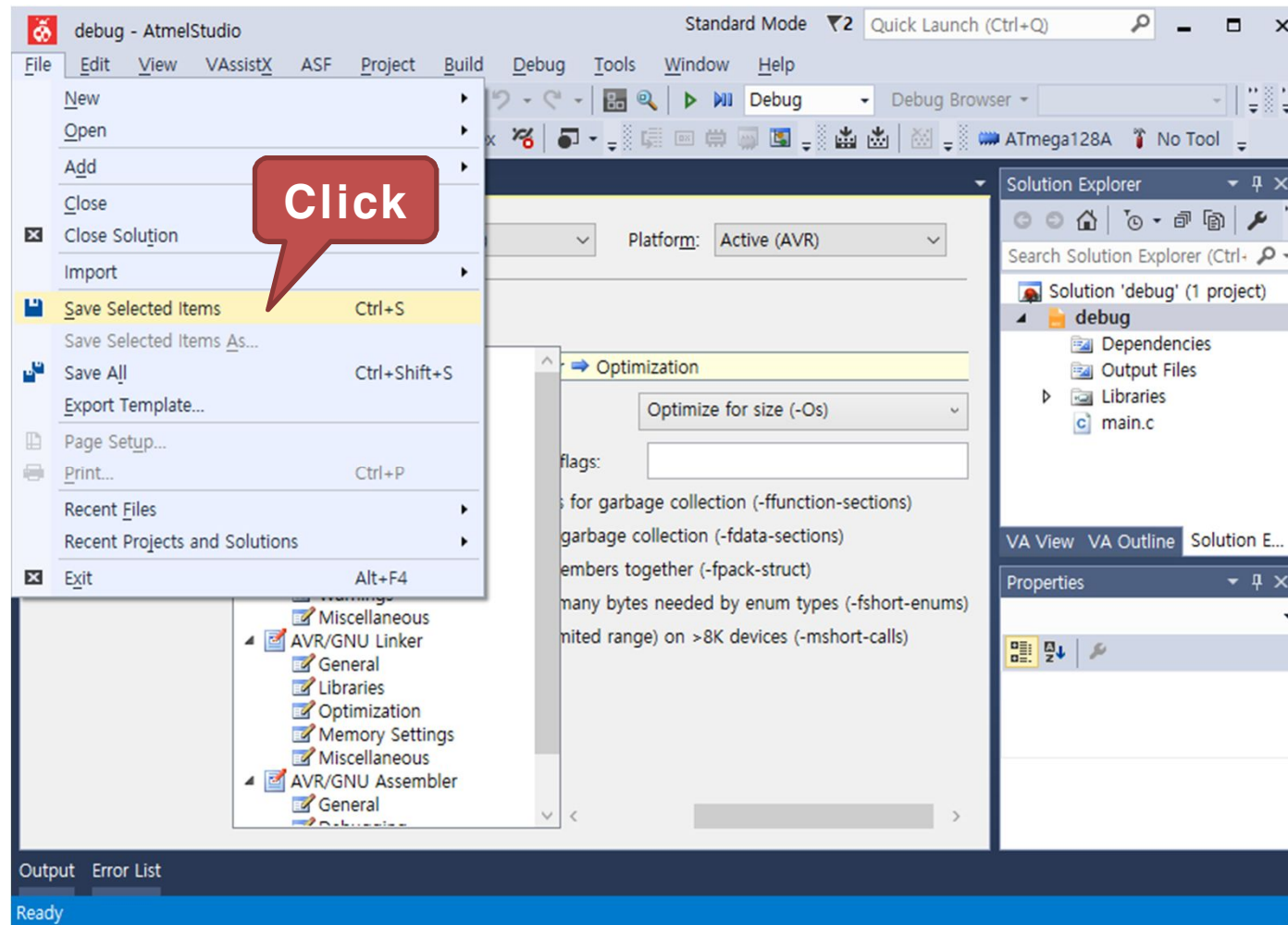
Atmel Studio 7.0 디버깅 모드

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "None (-O0)" 선택



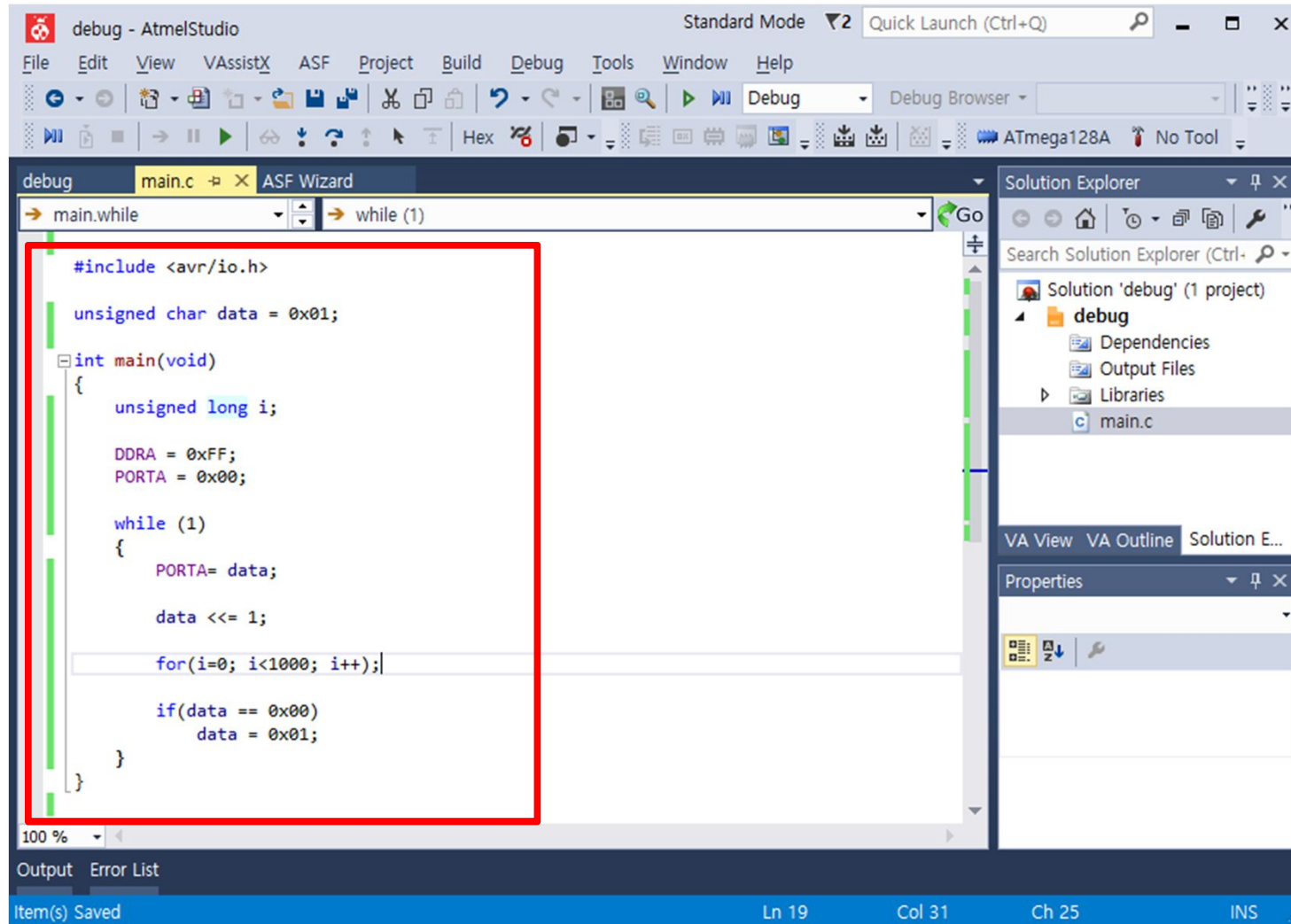
Atmel Studio 7.0 디버깅 모드

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



Atmel Studio 7.0 디버깅 모드

- main.c 파일을 선택한 후 소스코드를 작성



Atmel Studio 7.0 디버깅 모드

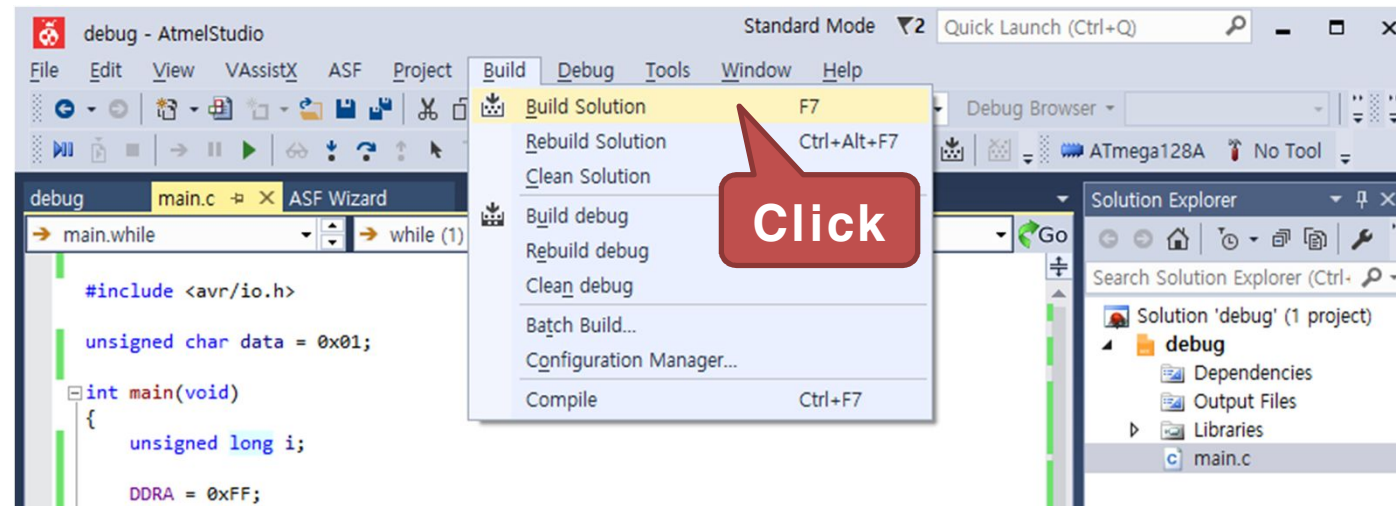
- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>

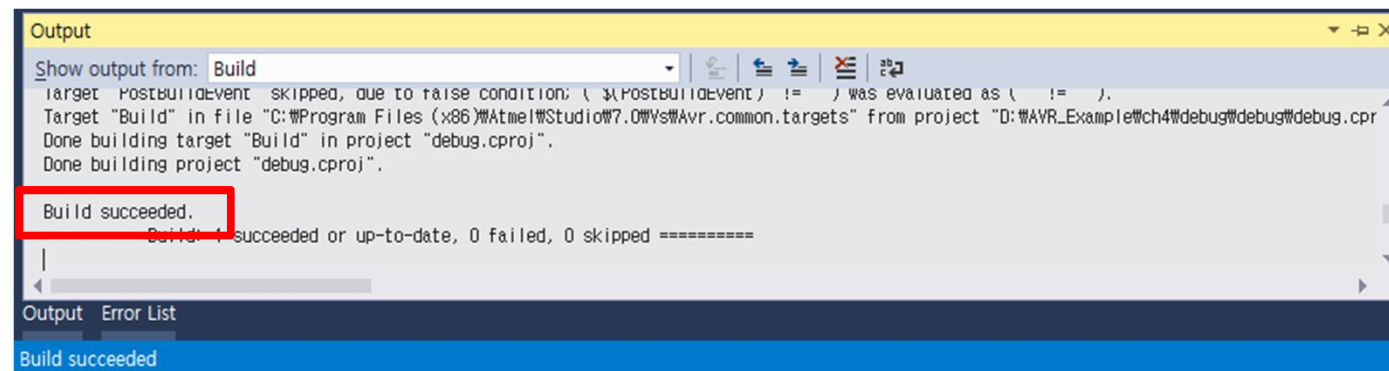
unsigned char data = 0x01;
int main(void) {
    unsigned long i;
    DDRA = 0xFF;
    PORTA = 0x00;
    while(1) {
        PORTA = data;
        data <<=1;
        for(i=0; i<1000; i++);
        if(data == 0x00)
            data = 0x01;
    }
}
```


Atmel Studio 7.0 디버깅 모드

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

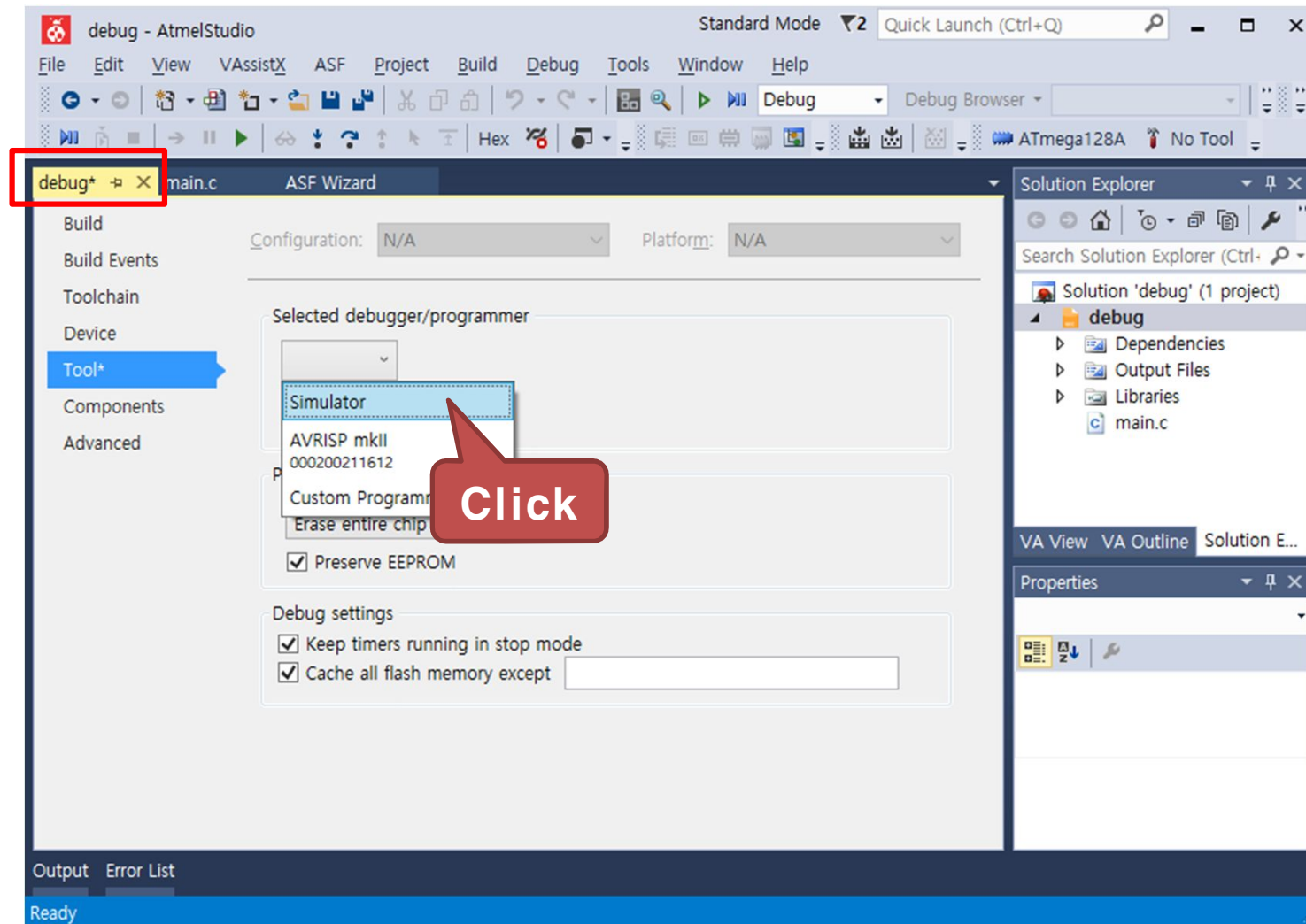


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



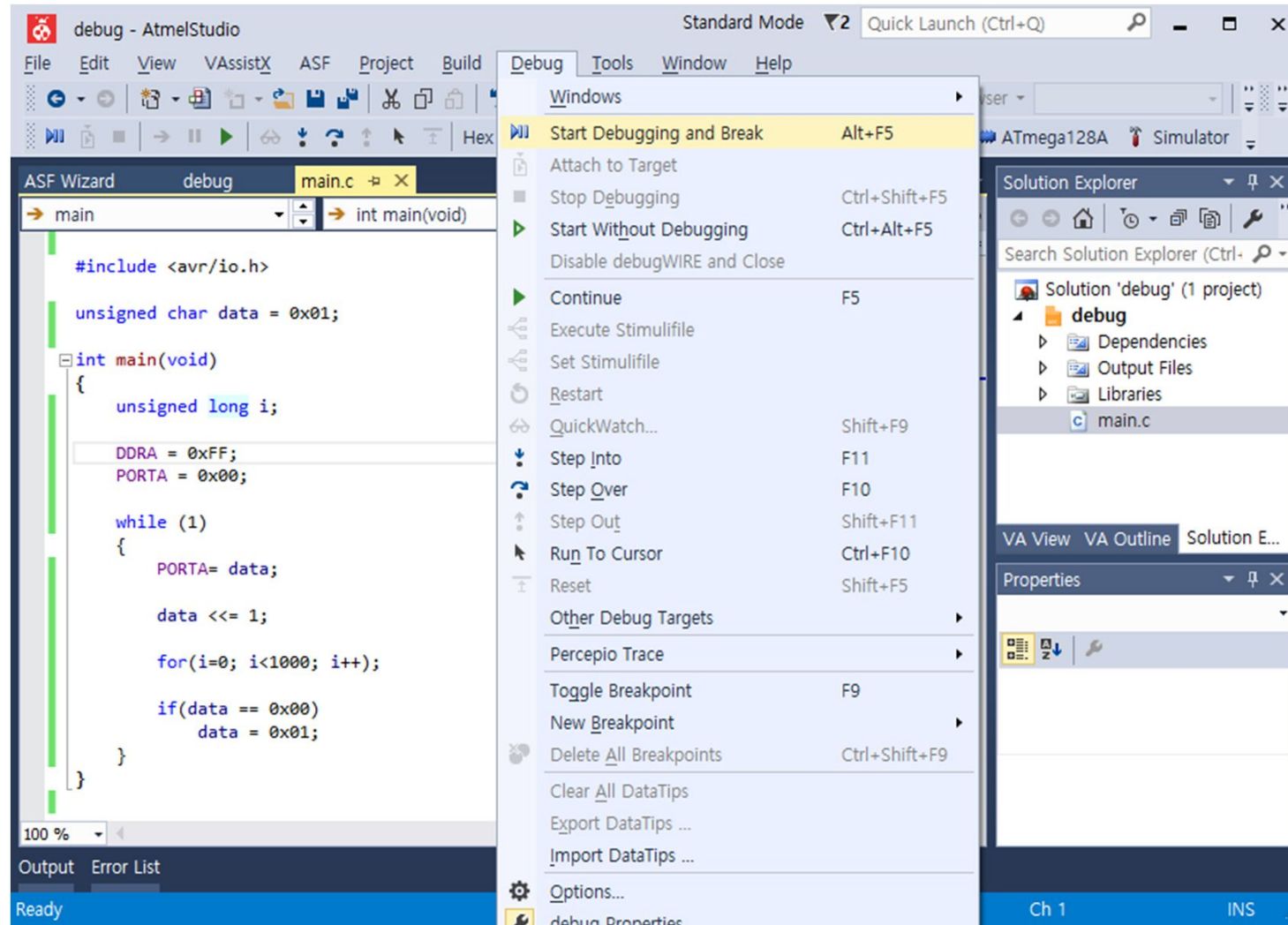
Atmel Studio 7.0 디버깅 모드

- debug Properties 창을 선택한 후 Tool 을 "Simulator"로 선택



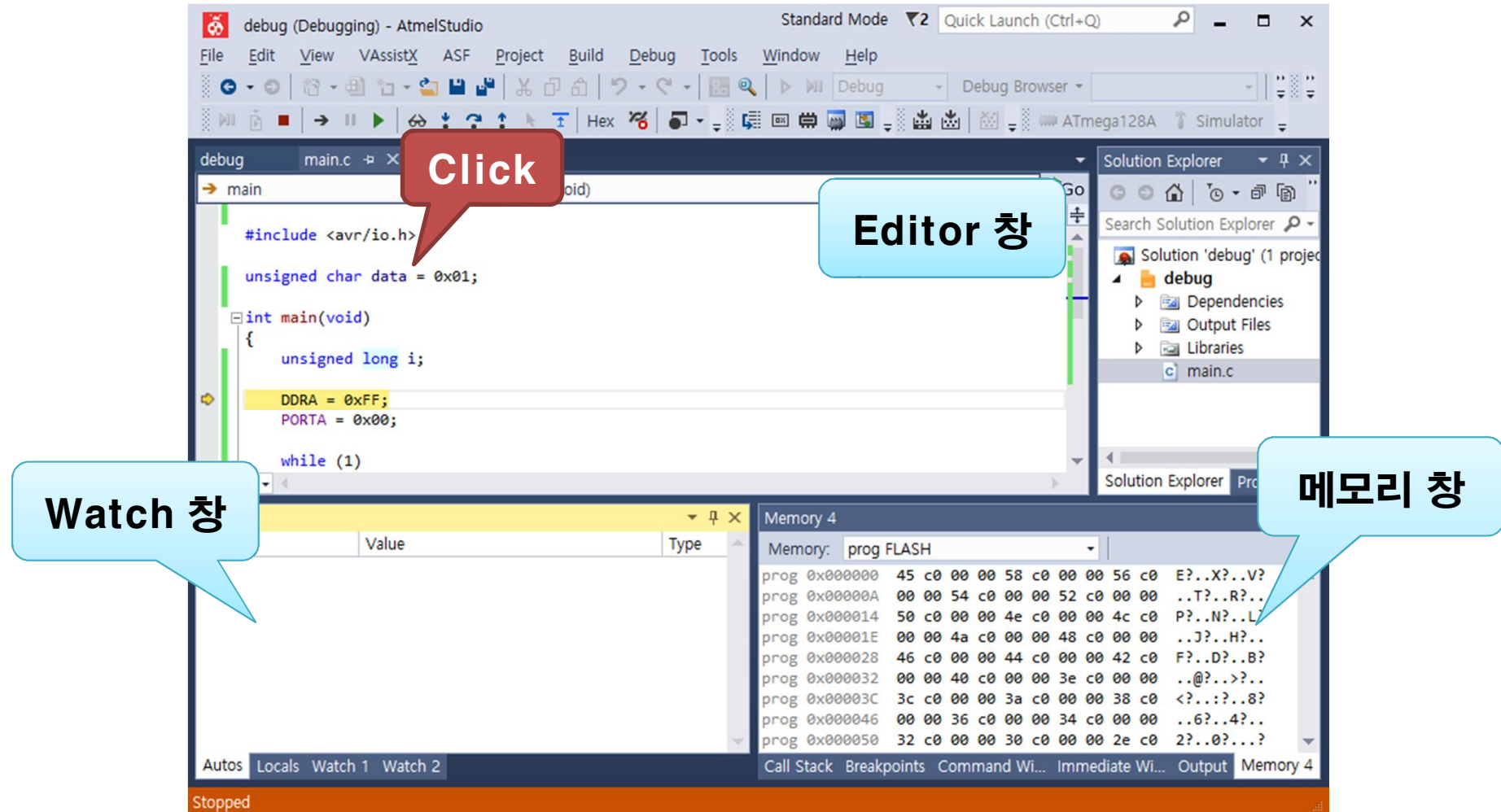
Atmel Studio 7.0 디버깅 모드

- Debug 탭을 선택한 후 "Start Debugging and Break"를 선택한다.



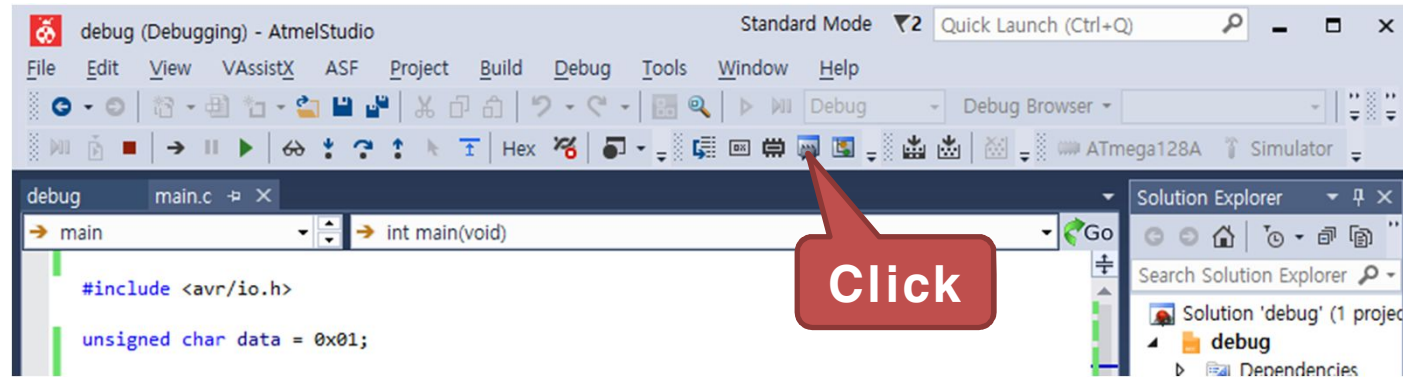
Atmel Studio 7.0 디버깅 모드

- 디버깅 모드 화면



Atmel Studio 7.0 디버깅 모드

- 다음과 같이 Processor Status 버튼을 클릭하면 주파수를 변경 가능



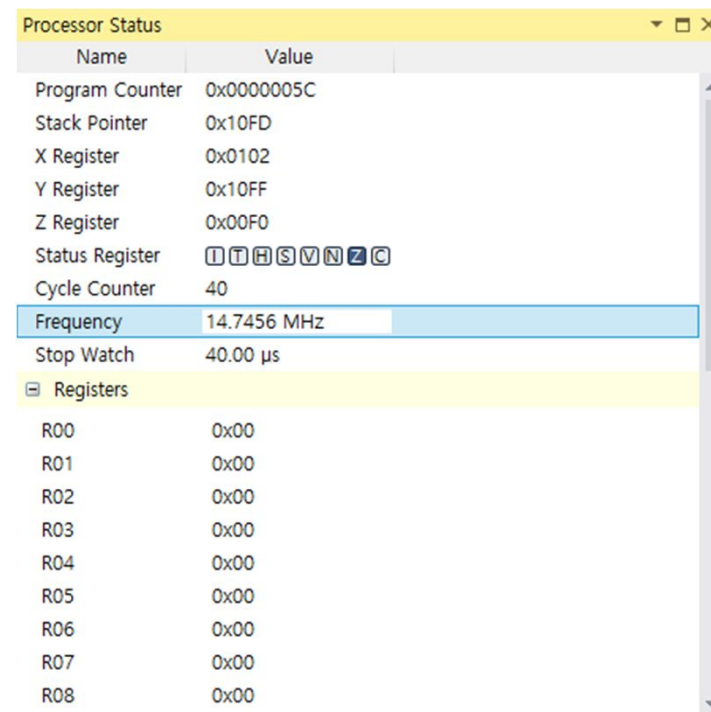
Processor Status	
Name	Value
Program Counter	0x0000005C
Stack Pointer	0x10FD
X Register	0x0102
Y Register	0x10FF
Z Register	0x00F0
Status Register	00000000
Cycle Counter	40
Frequency	1.000 MHz
Stop Watch	40.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00



Processor Status	
Name	Value
Program Counter	0x0000005C
Stack Pointer	0x10FD
X Register	0x0102
Y Register	0x10FF
Z Register	0x00F0
Status Register	00000000
Cycle Counter	40
Frequency	14.7456 MHz
Stop Watch	40.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00

Atmel Studio 7.0 디버깅 모드

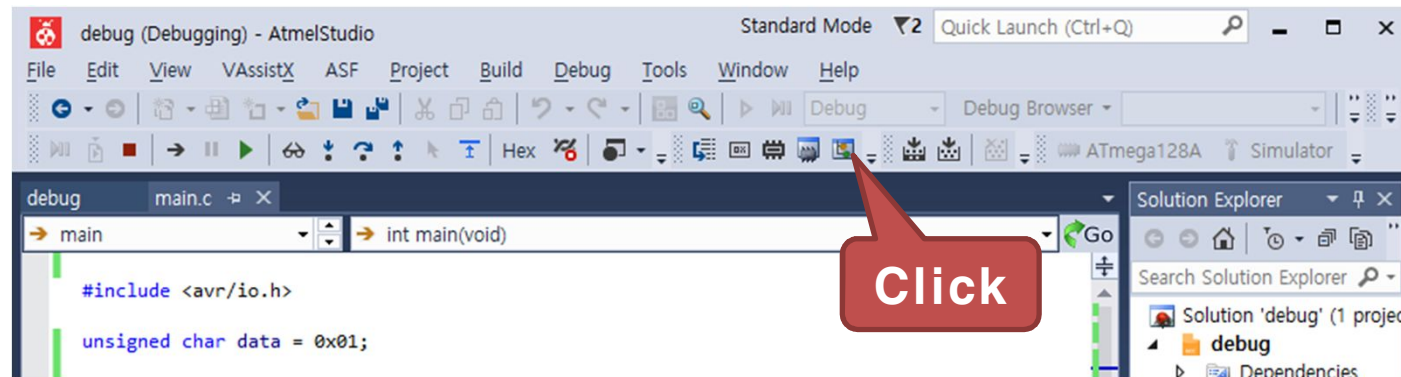
- Processor Status 창
 - AVR 내부 레지스터 값을 표시
 - 프로그램 카운터와 스택 포인터, 범용 레지스터의 값 확인 가능



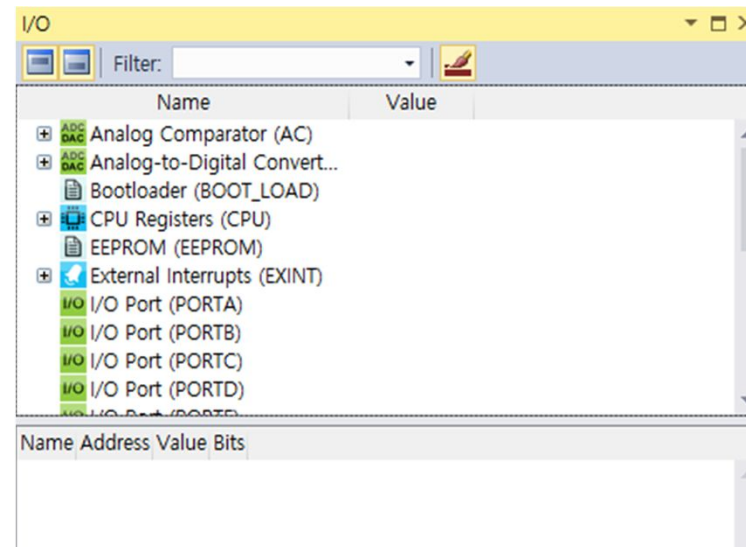
Name	Value
Program Counter	0x0000005C
Stack Pointer	0x10FD
X Register	0x0102
Y Register	0x10FF
Z Register	0x00F0
Status Register	I T H S V N Z C
Cycle Counter	40
Frequency	14.7456 MHz
Stop Watch	40.00 µs
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00

Atmel Studio 7.0 디버깅 모드

- 다음과 같이 I/O 버튼을 클릭하면 I/O View 창이 나타남

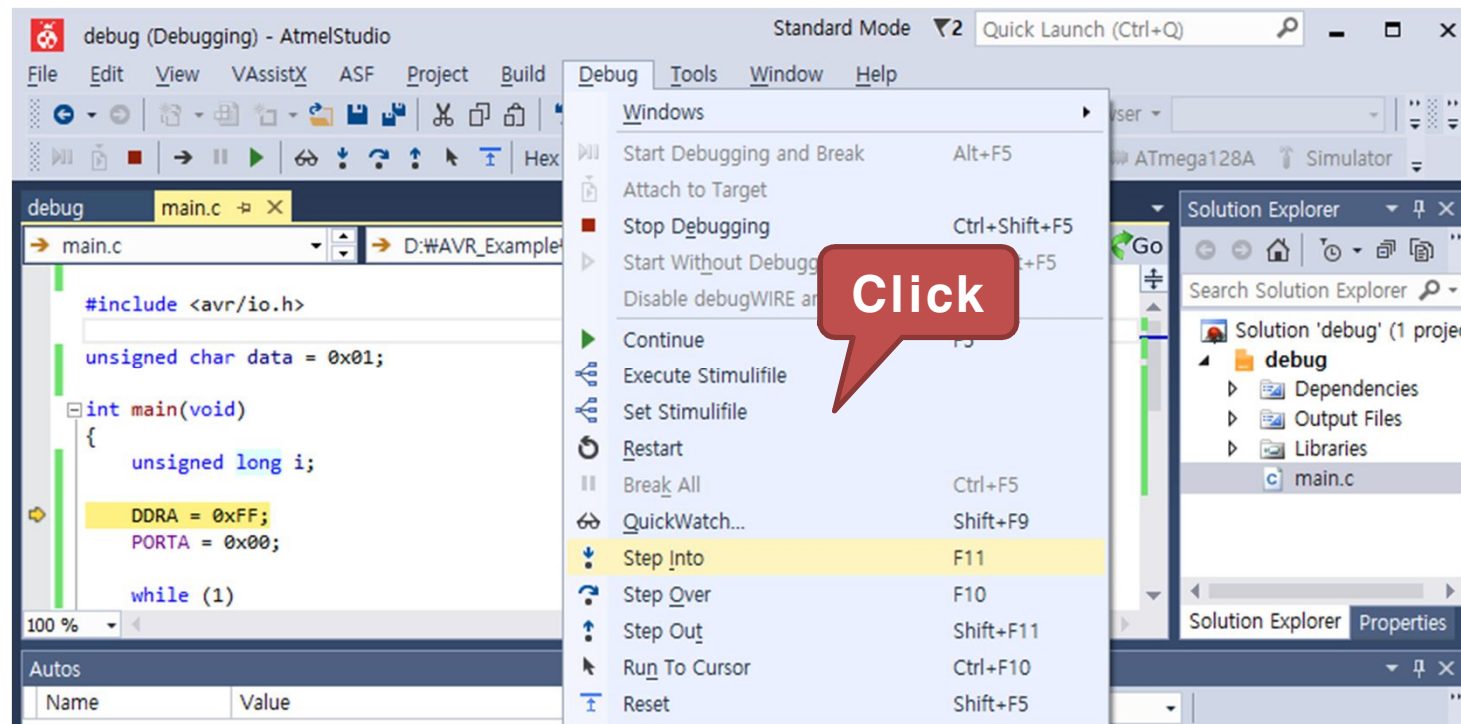


- 디버깅 모드 화면 : I/O View 창
 - 입출력 레지스터 및 타이머 등의 레지스터 값을 표시
 - 포트A~G 레지스터 및 타이머, UART 레지스터가 표시



Atmel Studio 7.0 디버깅 모드

- 프로그램을 실행하여 레지스터 값 변화를 관찰
- 먼저 Debug 탭의 Step Into 버튼을 눌러 프로그램 명령 한 줄을 실행 또는 키보드의 "F11"을 누름



Atmel Studio 7.0 디버깅 모드

```
#include <avr/io.h>

unsigned char data = 0x01;

int main(void)
{
    unsigned long i;

    DDRA = 0xFF;
    PORTA = 0x00;

    while (1)
```

Name	Address	Value	Bits
I/O PINA	0x39	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O DDRA	0x3A	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O PORTA	0x3B	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

I/O View 창

- Step Into(F11) 실행



```
#include <avr/io.h>

unsigned char data = 0x01;

int main(void)
{
    unsigned long i;

    DDRA = 0xFF;
    PORTA = 0x00;

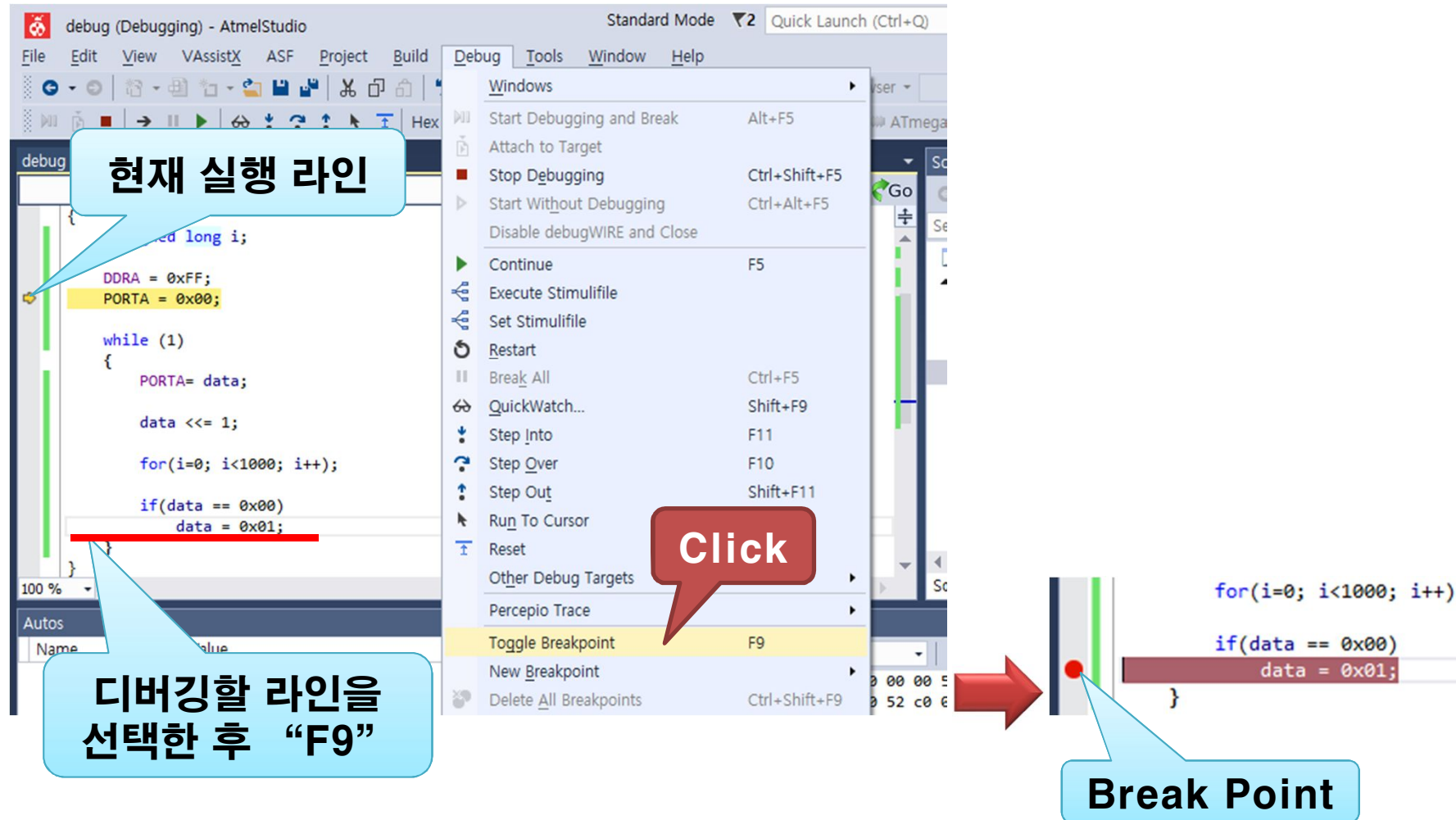
    while (1)
```

Name	Address	Value	Bits
I/O PINA	0x39	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O DDRA	0x3A	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O PORTA	0x3B	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

I/O View 창

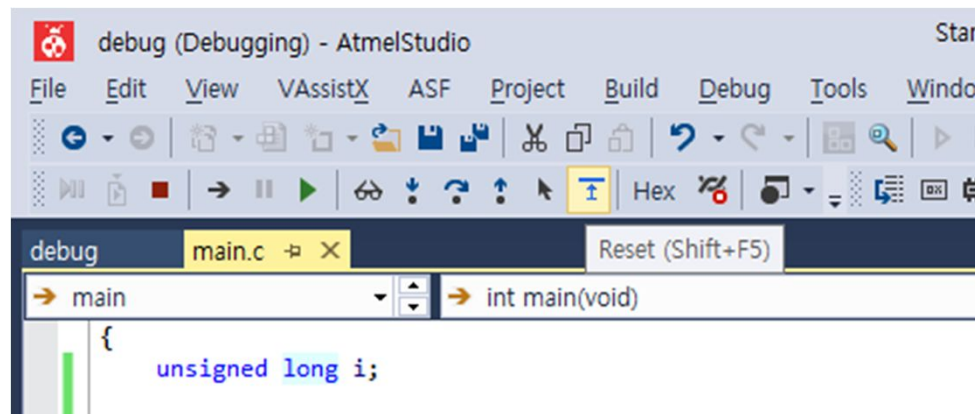
Atmel Studio 7.0 디버깅 모드

- Break Point
 - 디버깅을 해보고 싶은 부분부터 디버깅을 할 수 있게 해주는 기능



Atmel Studio 7.0 디버깅 모드

- Break Point
 - 프로그램 리셋(Debug -> Reset)

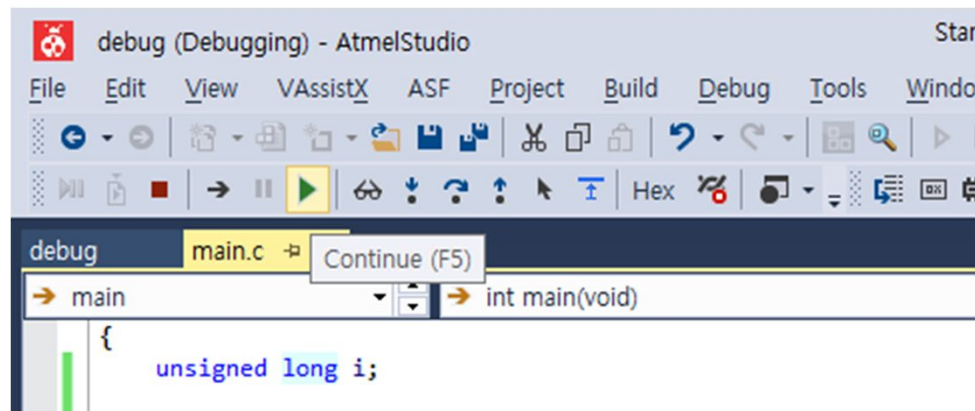


```
int main(void)
{
    unsigned long i;
    DDRA = 0xFF;
    PORTA = 0x00;

    while (1)
    {
```

**노란 화살표가
프로그램 시작점에 위치**

- Continue (Debug -> Continue)



```
data <= 1;
for(i=0; i<1000;
if(data == 0x00)
    data = 0x01;
}
```

**노란 화살표가
Break Point에 위치**

Atmel Studio 7.0 디버깅 모드

● Break Point 지정

```

{
    PORTA= data;

    data <= 1;

    for(i=0; i<1000; i++);

    if(data == 0x00)
        data = 0x01;
}

```

Name	Address	Value	Bits
I/O PINA	0x39	0x80	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRA	0x3A	0xFF	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O PORTA	0x3B	0x80	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

● Step Into(F11) 실행

```

{
    PORTA= data;

    data <= 1;

    for(i=0; i<1000; i++);

    if(data == 0x00)
        data = 0x01;
}

```

Name	Address	Value	Bits
I/O PINA	0x39	0x80	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRA	0x3A	0xFF	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O PORTA	0x3B	0x80	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

● Step Into(F11) 실행

```

{
    PORTA= data;

    data <= 1;

    for(i=0; i<1000; i++);

    if(data == 0x00)
        data = 0x01;
}

```

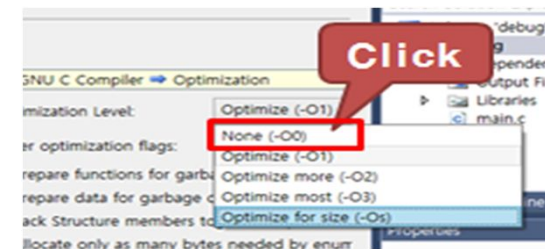
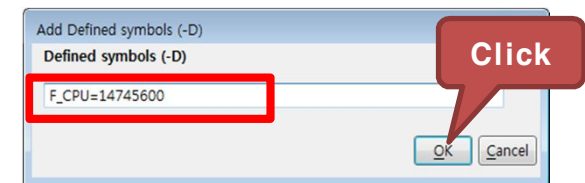
Name	Address	Value	Bits
I/O PINA	0x39	0x80	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRA	0x3A	0xFF	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O PORTA	0x3B	0x01	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☒ ☐

내부 메모리와 레지스터 이해

- 실습 개요
 - 하나의 예제 프로그램 작성
 - Atmel Studio의 메모리 Dump 및 변수 확인 창 등을 활용하여 내부 메모리와 레지스터의 변화를 관찰
- 실습 목표
 - 내부 메모리의 동작 방식을 이해
 - 프로그램이 실행되면서 내부 메모리들을 어떻게 사용하는지를 이해

내부 메모리와 레지스터 이해

- 예제 프로그램 작성
 - Atmel Studio 실행
 - New Project 생성
 - Name : 04_MEMORY_Example, Location : D:\WAVR_Example
 - Device Selection : ATmega128A
 - 프로젝트 설정
 - Project 탭에서 "04_MEMORY_Example Properties..." 선택
 - Toolchain -> AVR/GNU C Compiler에서
 - Symbols -> F_CPU=14745600 추가
 - Optimization -> None (-O0) 선택
 - 저장 (Ctrl+S)
 - 소스코드 작성



내부 메모리와 레지스터 이해

- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>

unsigned char A = 0xAA;
unsigned char B = 0xBB;
unsigned char C = 0xCC;
unsigned char *ADD = 0x130;

int main(void) {
    DDRA = 0xFF;
    PORTA = 0x00;
    DDRF = 0xFF;
    PORTF = 0x00;
```

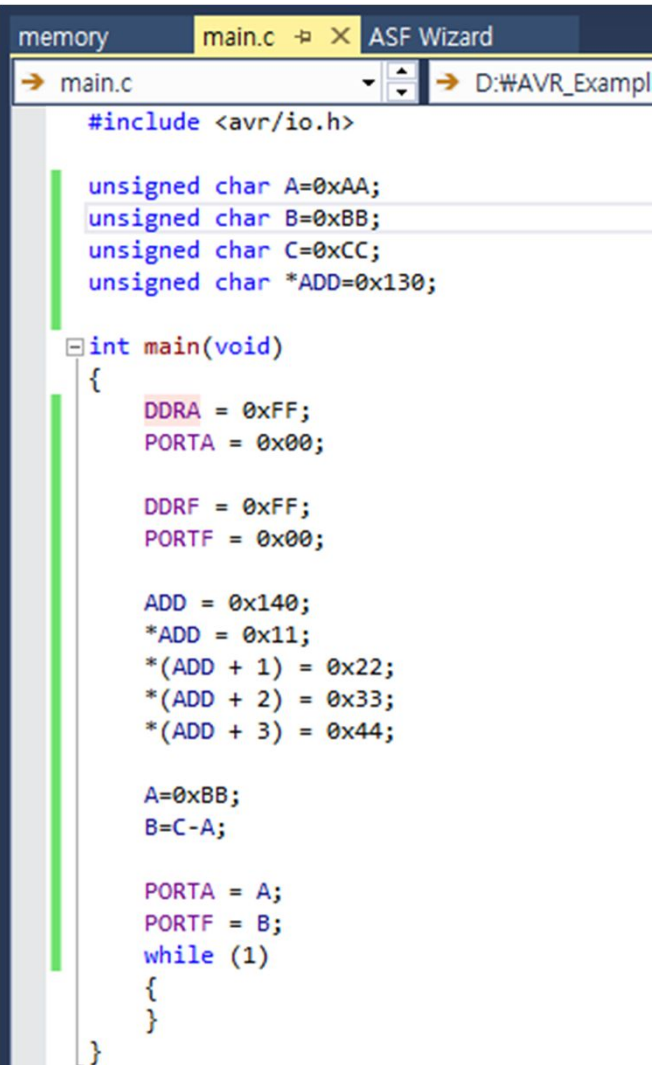

내부 메모리와 레지스터 이해

- 구동 프로그램
 - main.c 코드 작성

```
ADD = 0x140;  
*ADD = 0x11;  
*(ADD + 1) = 0x22;  
*(ADD + 2) = 0x33;  
*(ADD + 3) = 0x44;  
  
A = 0xBB;  
B = C - A;  
  
PORTA = A;  
PORTB = B;  
  
while(1) { }  
}
```

내부 메모리와 레지스터 이해

- 소스 분석(변수 선언)
 - A,B,C : 전역 변수
 - 프로그램 시작과 함께 일정 영역을 할당
 - 할당 받은 영역에 초기값 저장 (0xAA, 0xBB, 0xCC)
 - Unsigned char이므로 1byte 할당
 - ADD : 포인터 변수
 - 역시 내부 일정 영역 할당
 - 메모리 영역에 데이터가 아닌 주소를 저장(0x130)
 - 0x130은 SRAM의 영역



```
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

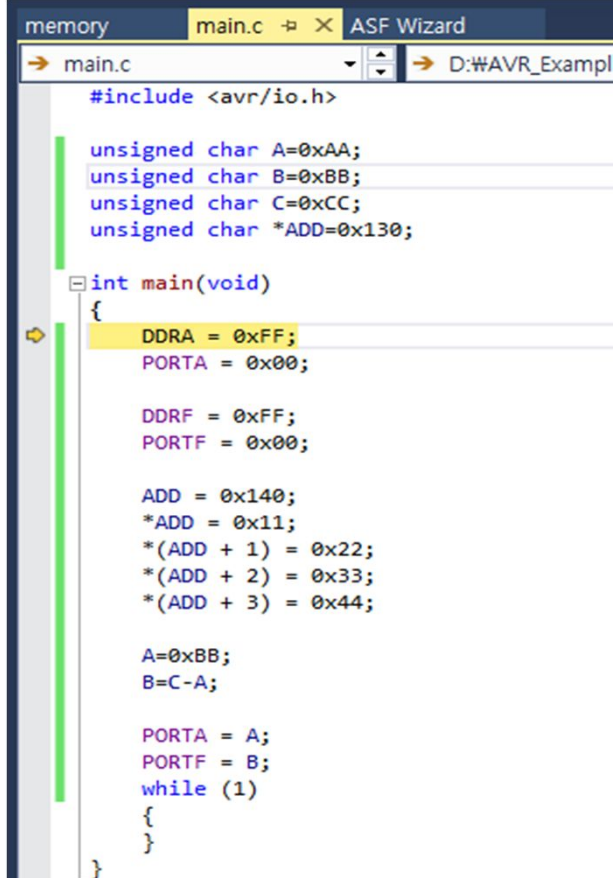
    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
```

내부 메모리와 레지스터 이해

- 프로그램 실행
 - 프로그램을 빌드
 - Buil 탭에서 "Build Solution" 클릭
 - Memory Properties 선택, Tool을 "Simulator"로 선택
 - 디버깅 모드 시작
 - 노란 화살표는 시작점에 놓임



```
memory  main.c  ASF Wizard
main.c
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

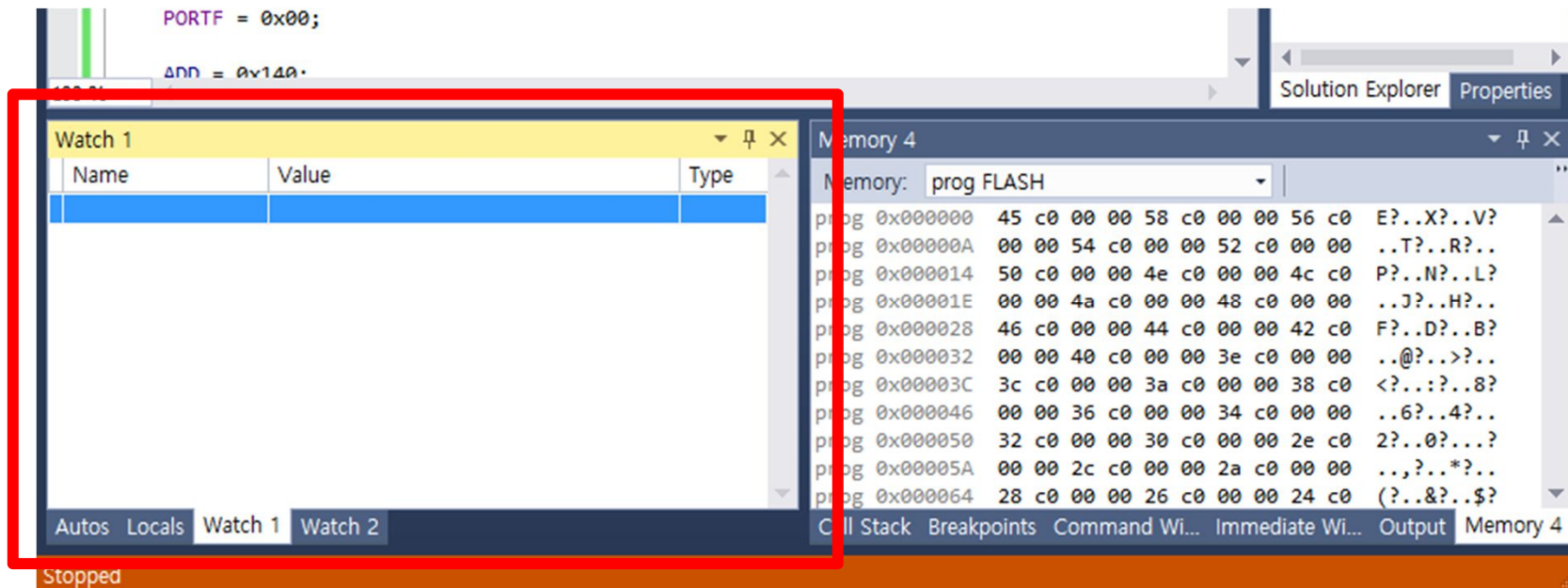
    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}
```

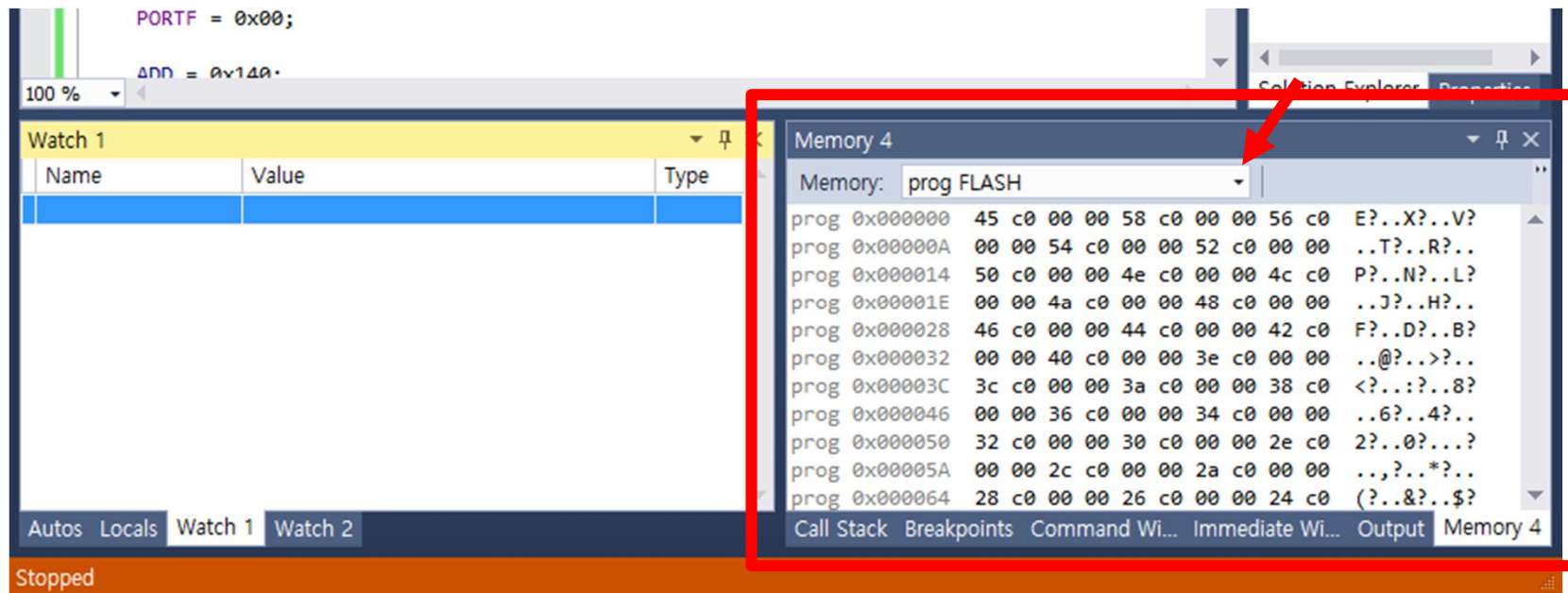
내부 메모리와 레지스터 이해

- 변수관찰 창(watch)
 - Atmel Studio의 좌측 하단에 위치



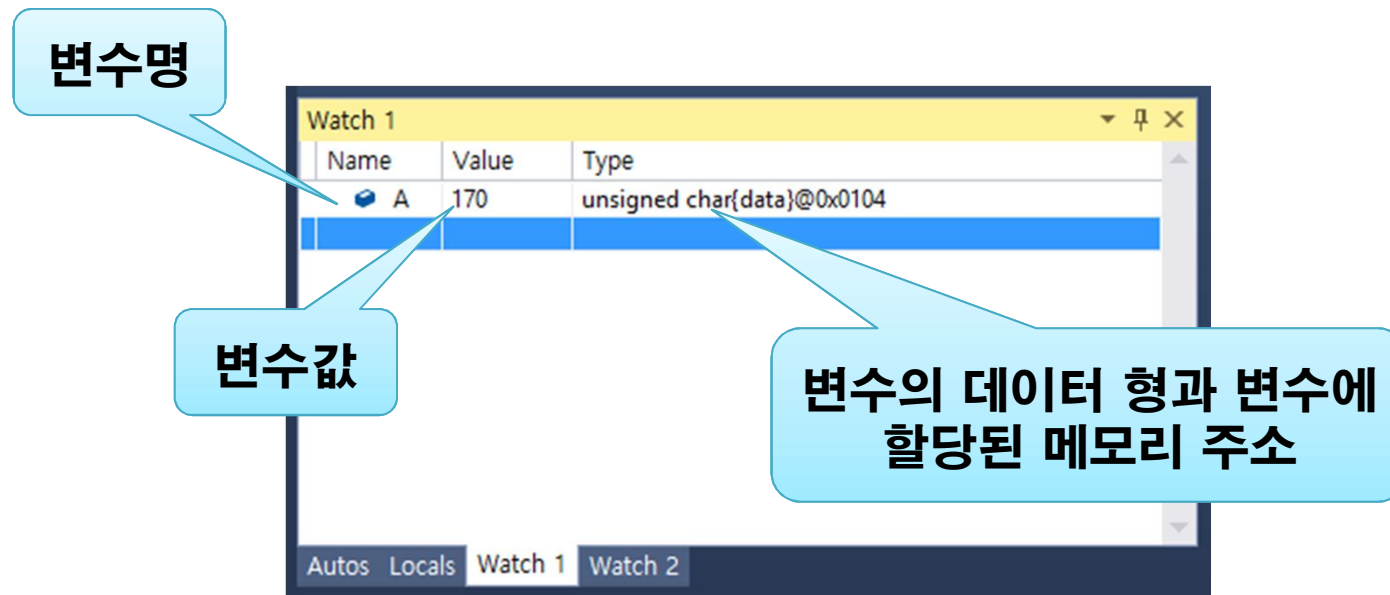
내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - Atmel Studio의 우측 하단에 위치



내부 메모리와 레지스터 이해

- 변수 관찰 창
 - 변수 관찰 창의 Name부분 빈칸을 더블 클릭하여 변수 이름을 적음
 - 여기서는 'A'를 넣음



내부 메모리와 레지스터 이해

- 변수 관찰 창
 - 같은 방법으로 B와 C를 입력

Watch 1		
Name	Value	Type
A	170	unsigned char{data}@0x0104
B	187	unsigned char{data}@0x0103
C	204	unsigned char{data}@0x0102

Value를
헥사값으로 표시

Watch 1		
Name	Value	Type
A	170	unsigned char{data}@0x0104
B	187	unsigned char{data}@0x0103
C	204	unsigned char{data}@0x0102

Click

Copy

Ctrl+C

Paste

Ctrl+V

Edit Value

Copy Value

Add Watch

Add Parallel Watch

Delete Watch

Select All

Clear All

Hexadecimal Display

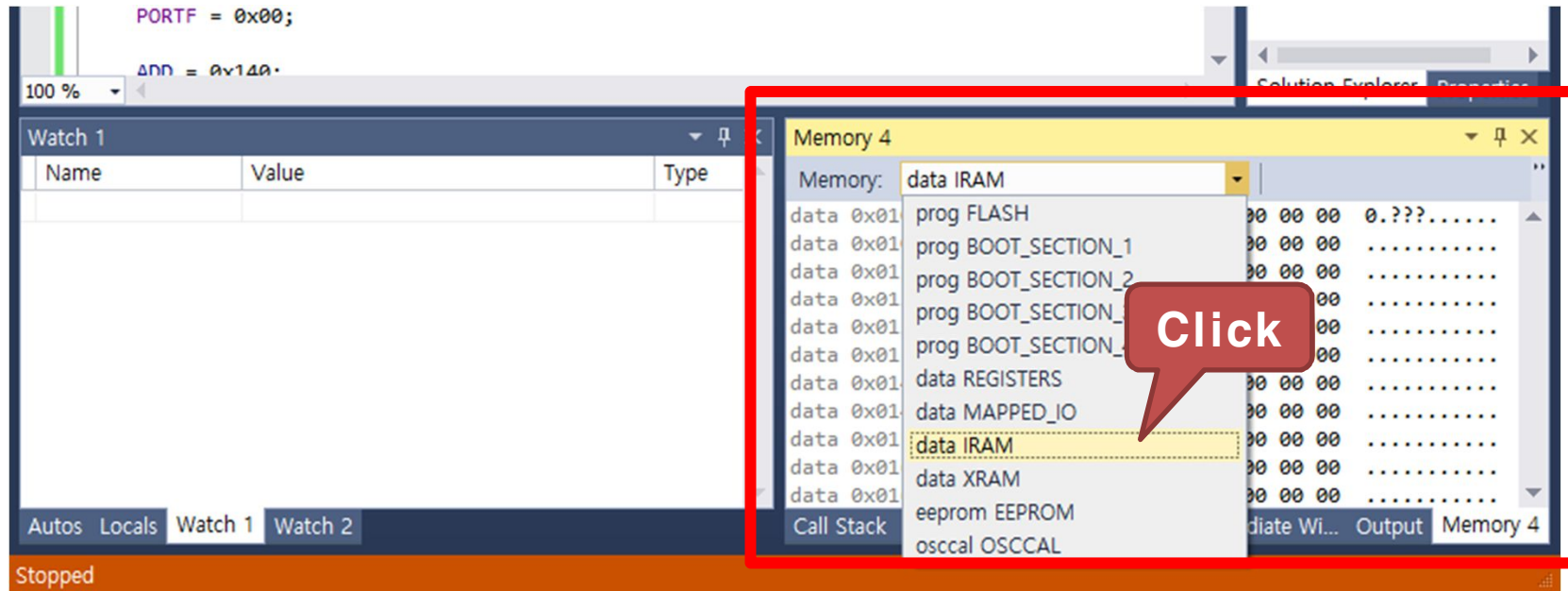
Collapse Parent

Go To Source Code

Go To Disassembly

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM선택
 - 데이터 메모리 영역을 보여줌



내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - 변수 'A', 'B', 'C' 의 위치와 저장된 값을 확인

The screenshot shows the 'Memory 4' window with the 'Memory' dropdown set to 'data IRAM'. The memory dump displays addresses from 0x0100 to 0x0178. Three callouts are present:

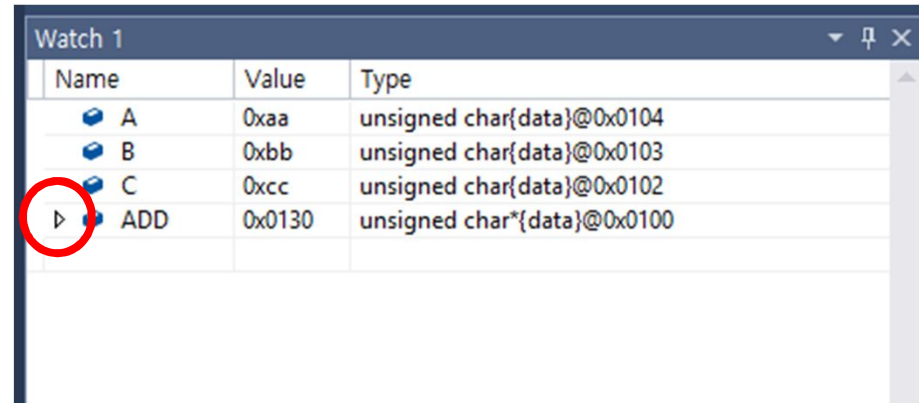
- 변수 B 영역 0x0103번지** (Variable B area, address 0x0103)
- 변수 A 영역 0x0104번지** (Variable A area, address 0x0104)
- 변수 C 영역 0x0102번지** (Variable C area, address 0x0102)

The memory dump shows the following data:

Address	Value (Hex)	Value (ASCII)
0x0100	30 01 cc bb aa 00 00 00 00 00 00 00 00	0.???.....
0x010C	00 00 00 00 00 00 00 00 00 00 00 00	
0x0118	00 00 00 00 00 00 00 00 00 00 00 00	
0x0124	00 00 00 00 00 00 00 00 00 00 00 00	
0x0130	00 00 00 00 00 00 00 00 00 00 00 00	
0x013C	00 00 00 00 00 00 00 00 00 00 00 00	
0x0148	00 00 00 00 00 00 00 00 00 00 00 00	
0x0154	00 00 00 00 00 00 00 00 00 00 00 00	
0x0160	00 00 00 00 00 00 00 00 00 00 00 00	
0x016C	00 00 00 00 00 00 00 00 00 00 00 00	
0x0178	00 00 00 00 00 00 00 00 00 00 00 00	

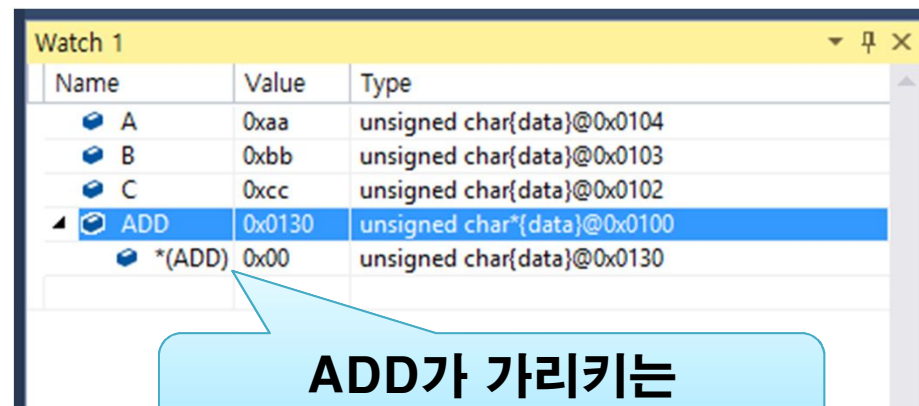
내부 메모리와 레지스터 이해

- 변수 관찰 창
 - 변수 관찰 창에 ADD 변수를 입력



Name	Value	Type
A	0xaa	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
▶ ADD	0x0130	unsigned char*(data)@0x0100

- ADD변수 왼쪽의 ▶기호를 누름

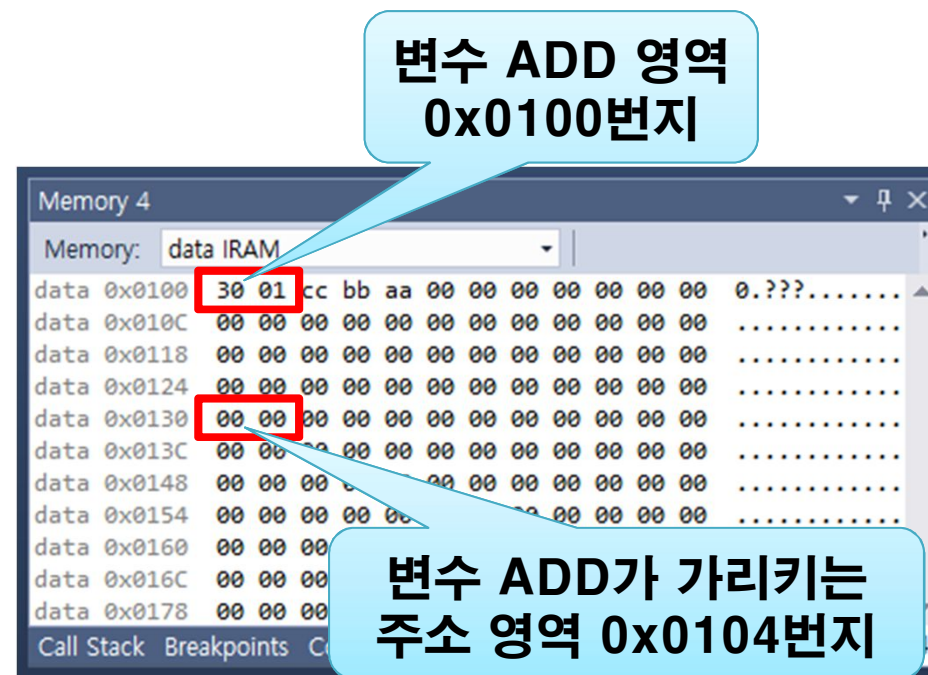


Name	Value	Type
A	0xaa	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
▲ ADD	0x0130	unsigned char*(data)@0x0100
*(ADD)	0x00	unsigned char{data}@0x0130

**ADD가 가리키는
메모리에 저장된 데이터 값**

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - 변수 "ADD" 의 위치와 저장된 값을 확인



내부 메모리와 레지스터 이해

● 프로그램 실행 전

```

memory    main.c  X ASF Wizard
→ main.c  D:\AVR_Exmpl

#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}

```

I/O

Filter:

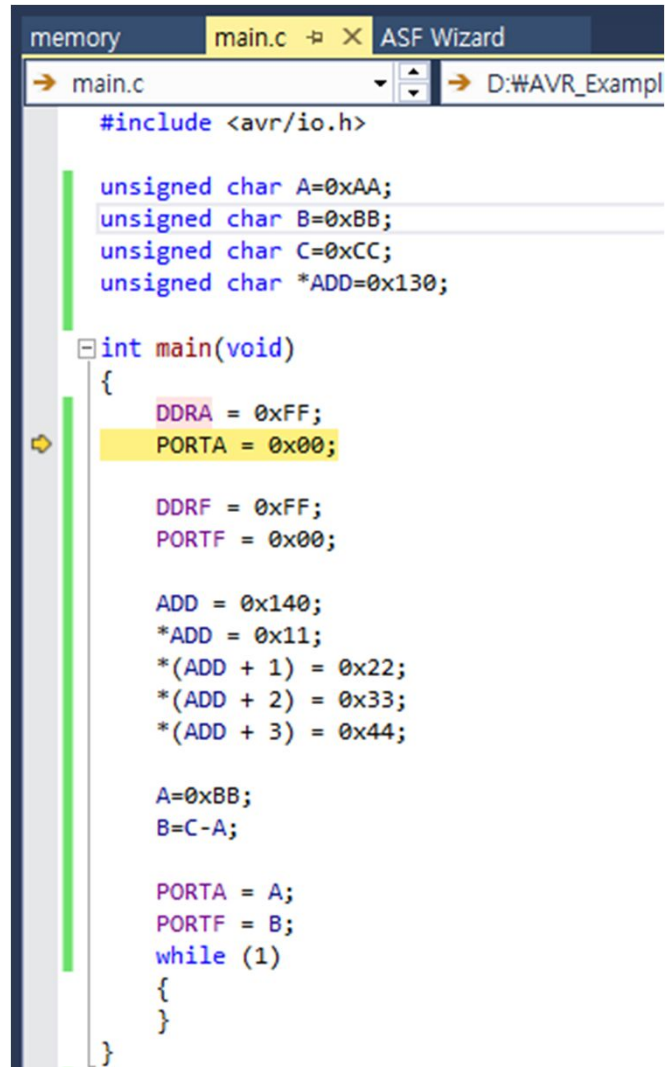
Name	Value
ADC (DAC)	
ADC (DAC)	
Bootloader (BOOT_LOAD)	
CPU Registers (CPU)	
EEPROM (EEPROM)	
External Interrupts (EXINT)	
I/O Port (PORTA)	
I/O Port (PORTB)	
I/O Port (PORTC)	
I/O Port (PORTD)	
I/O Port (PORTE)	

Name	Address	Value	Bits
PINA	0x39	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
DDRA	0x3A	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
PORTA	0x3B	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

PORTA 레지스터 관찰

내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)



```
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

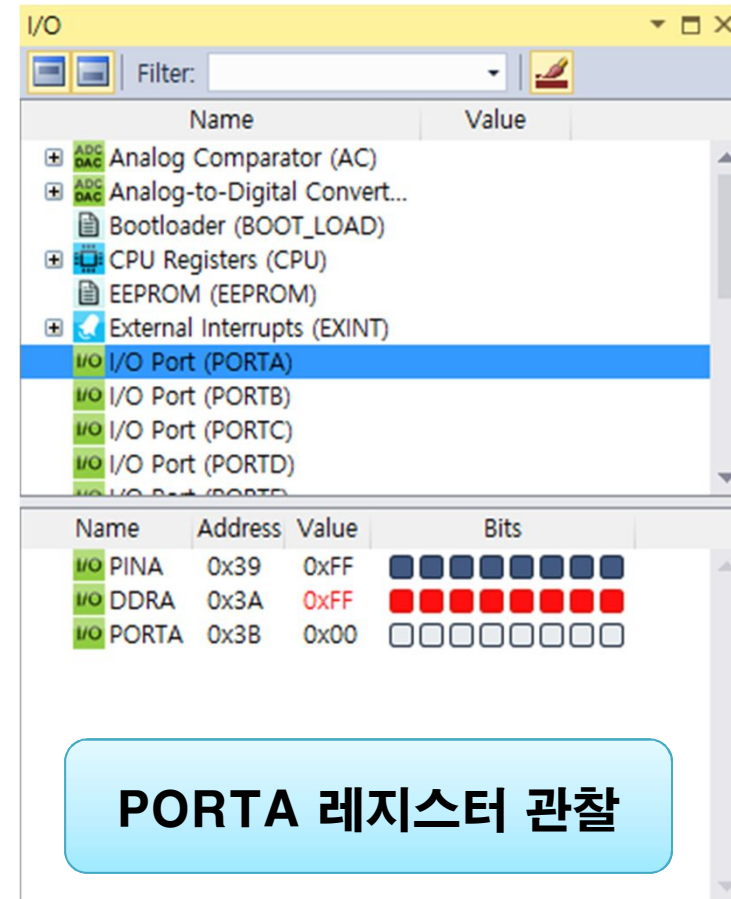
int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
```



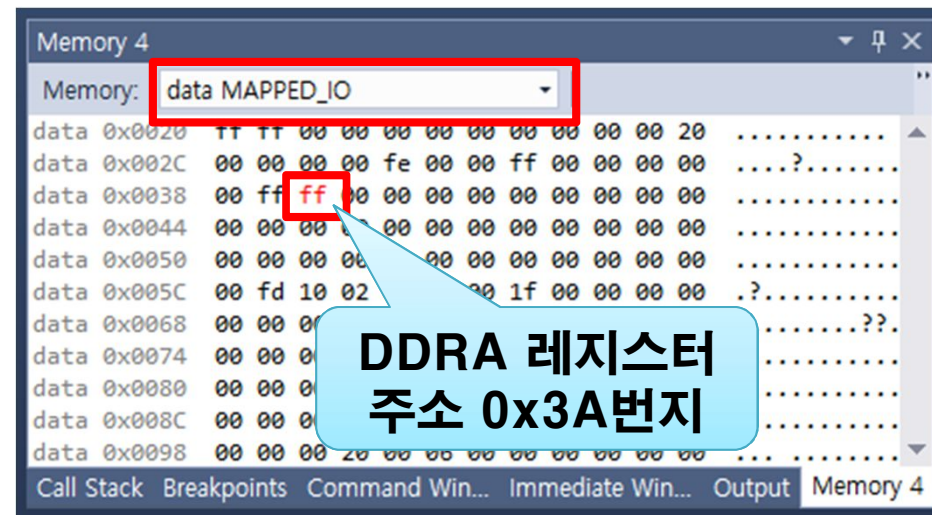
Name	Value
ADC (DAC)	
Analog Comparator (AC)	
Analog-to-Digital Convert...	
Bootloader (BOOT_LOAD)	
CPU Registers (CPU)	
EEPROM (EEPROM)	
External Interrupts (EXINT)	
I/O Port (PORTA)	
I/O Port (PORTB)	
I/O Port (PORTC)	
I/O Port (PORTD)	
I/O Port (PORTE)	

Name	Address	Value	Bits
PINA	0x39	0xFF	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
DDRA	0x3A	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
PORTA	0x3B	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

PORTA 레지스터 관찰

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data MAPPED_IO 선택



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11) 3회

```

memory    main.c  ASF Wizard
main.c    D:\AVR_Exmpl

#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}

```

I/O

Filter:

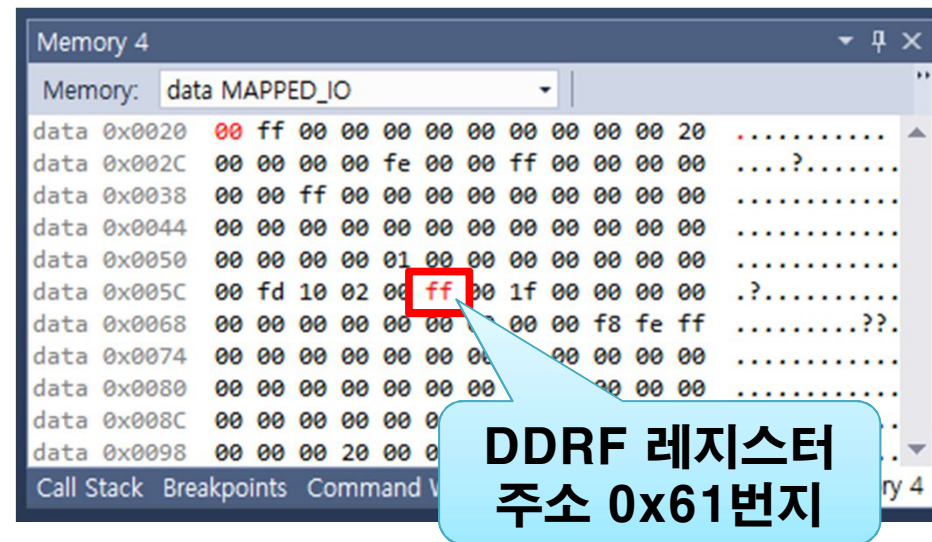
Name	Value
External Interrupts (EXTI)	
I/O Port (PORTA)	
I/O Port (PORTB)	
I/O Port (PORTC)	
I/O Port (PORTD)	
I/O Port (PORTE)	
I/O Port (PORTF)	
I/O Port (PORTG)	
JTAG Interface (JTAG)	
Other Registers (MISC)	
Serial Peripheral Interface...	

Name	Address	Value	Bits
I/O PINF	0x20	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRF	0x61	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O PORTF	0x62	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

PORTF 레지스터 관찰

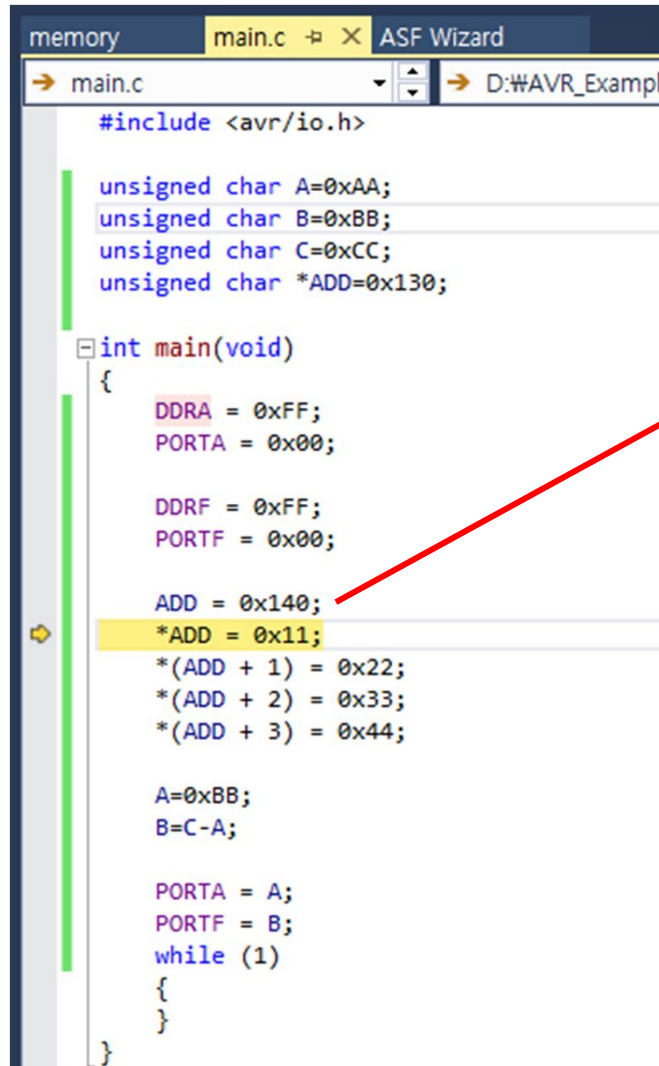
내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data MAPPED_IO 선택



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)



```
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

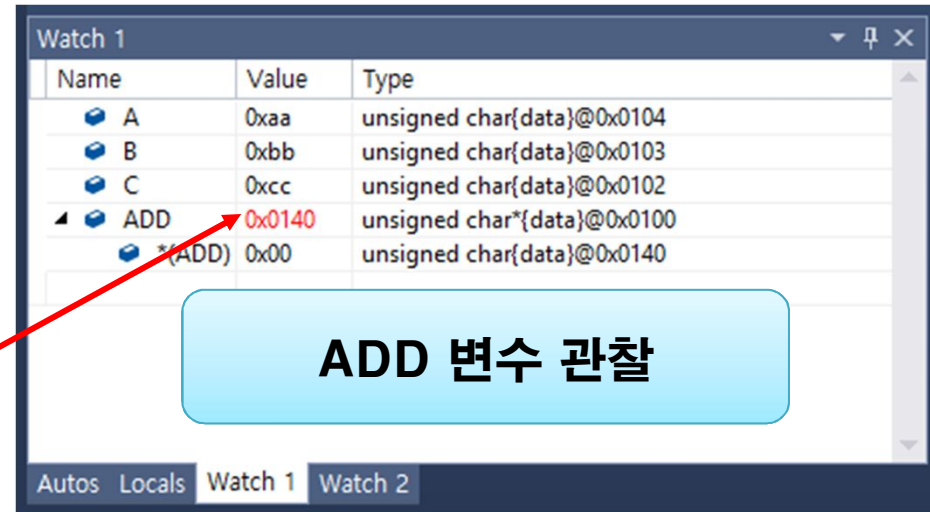
int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
```

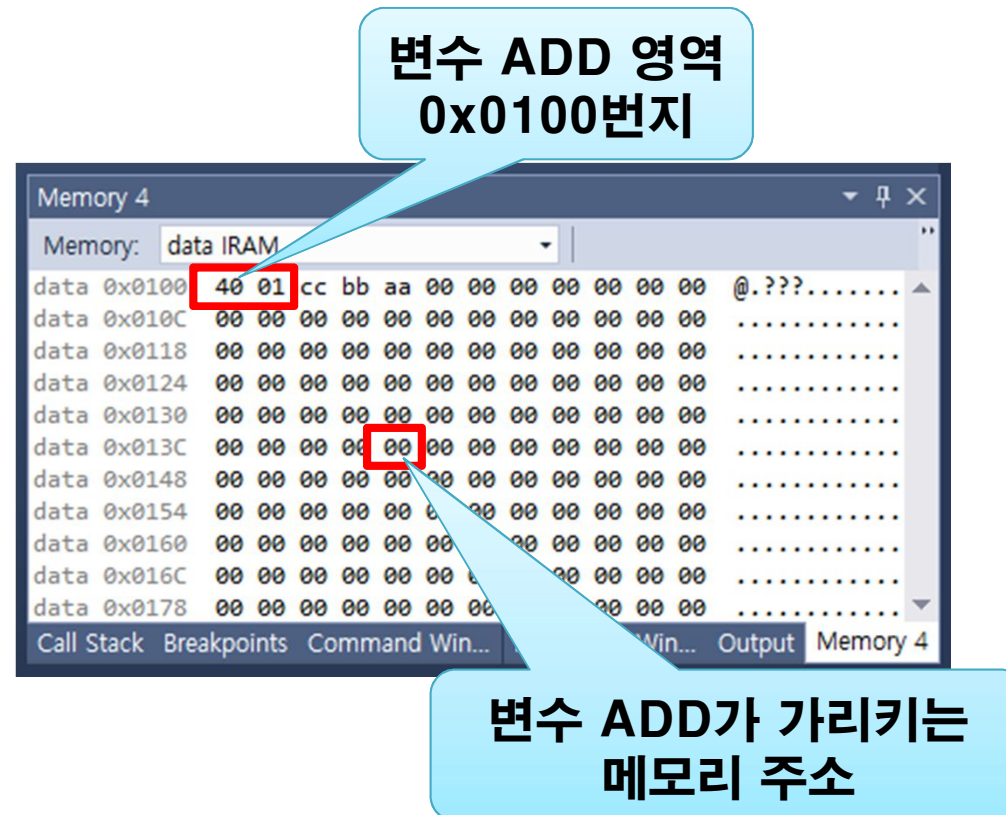


Name	Value	Type
A	0xaa	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
ADD	0x0140	unsigned char*(data)@0x0100
*(ADD)	0x00	unsigned char{data}@0x0140

ADD 변수 관찰

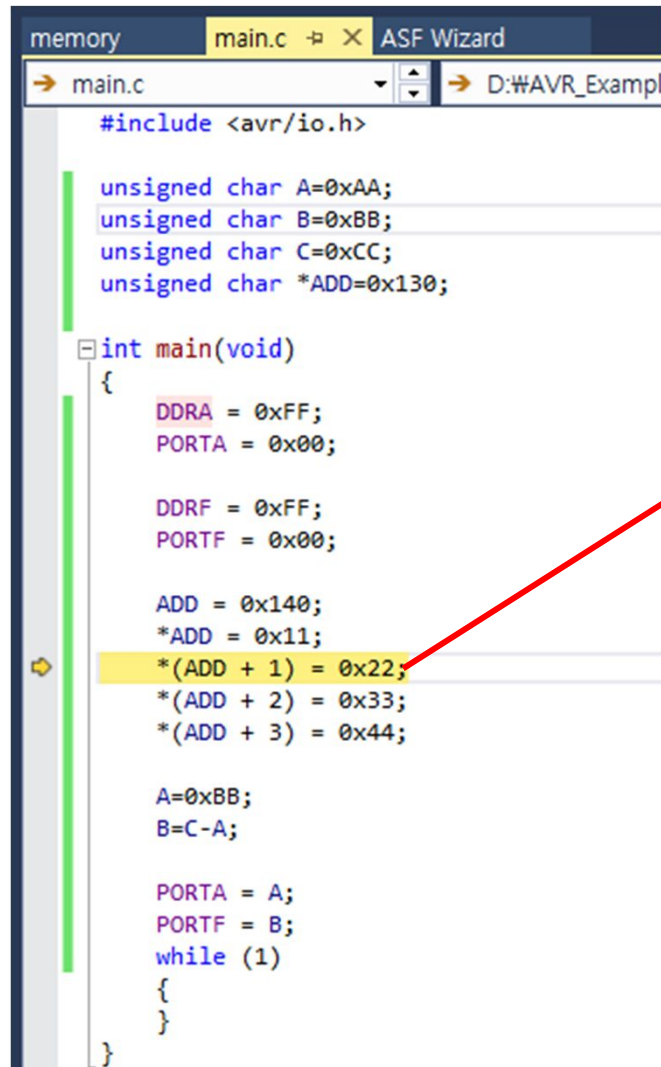
내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM 선택



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)



```
memory  main.c  ASF Wizard
main.c  D:\AVR_Exempl
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

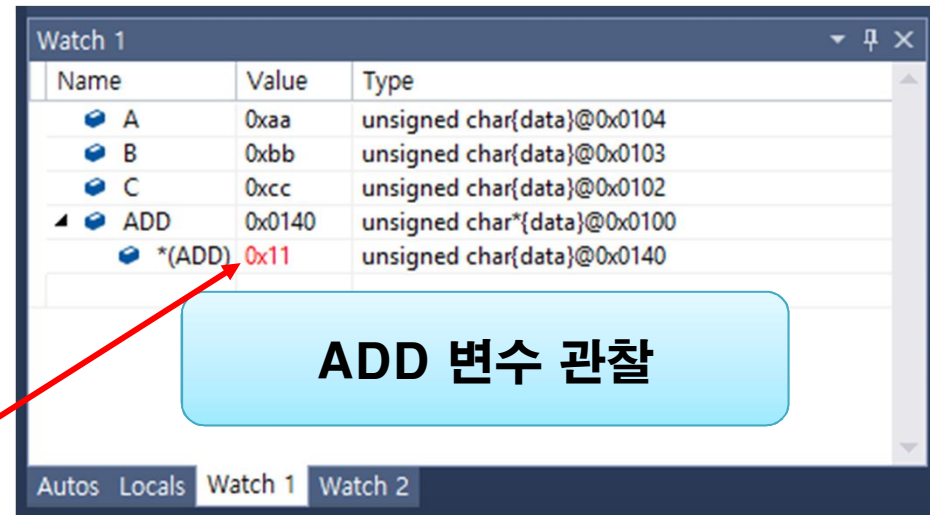
int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    * (ADD + 1) = 0x22;
    * (ADD + 2) = 0x33;
    * (ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}
```



Name	Value	Type
A	0xaa	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
ADD	0x0140	unsigned char*(data)@0x0100
* (ADD)	0x11	unsigned char{data}@0x0140

ADD 변수 관찰

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM 선택

변수 ADD 영역
0x0100번지

Memory 4

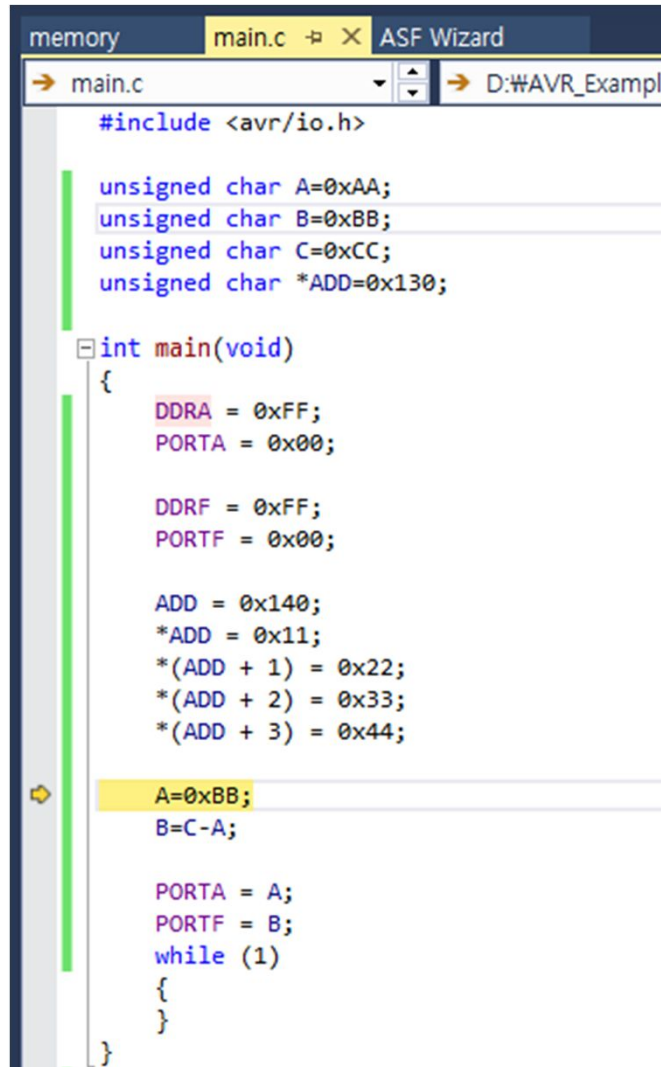
Memory: data IRAM

Address	Value
data 0x0100	40 01 cc bb aa 00 00 00 00 00 00 00 00 @.???
data 0x010C	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0118	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0124	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0130	00 00 00 00 00 00 00 00 00 00 00 00
data 0x013C	00 00 00 00 11 00 00 00 00 00 00 00
data 0x0148	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0154	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0160	00 00 00 00 00 00 00 00 00 00 00 00
data 0x016C	00 00 00 00 00 00 00 00 00 00 00 00
data 0x0178	00 00 00 00 00 00 00 00 00 00 00 00

변수 ADD가 가리키는
메모리 주소

내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11) 3회



```
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

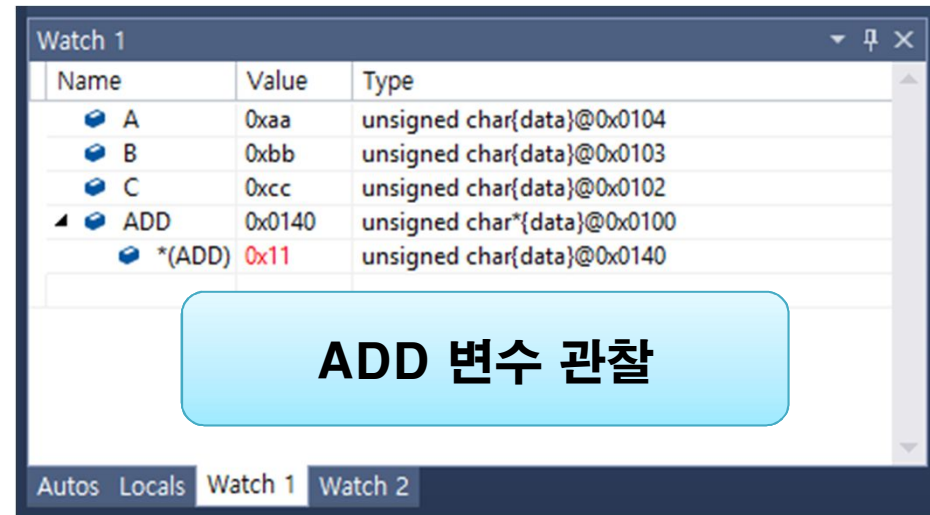
int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}
```



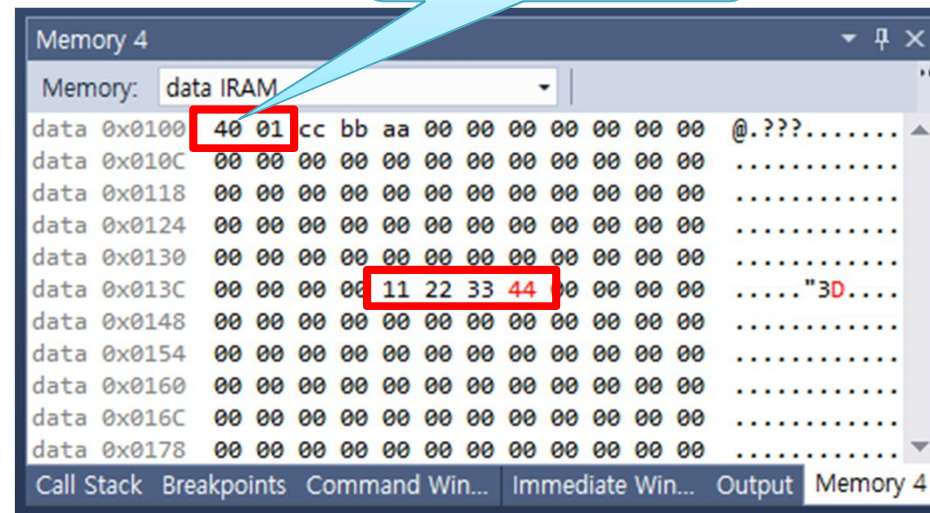
Name	Value	Type
A	0xaa	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
ADD	0x140	unsigned char{data}@0x0100
* (ADD)	0x11	unsigned char{data}@0x0140

ADD 변수 관찰

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM 선택

변수 ADD 영역
0x0100번지



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)

```

memory    main.c  ASF Wizard
→ main.c  D:\AVR_Exempl

#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}

```

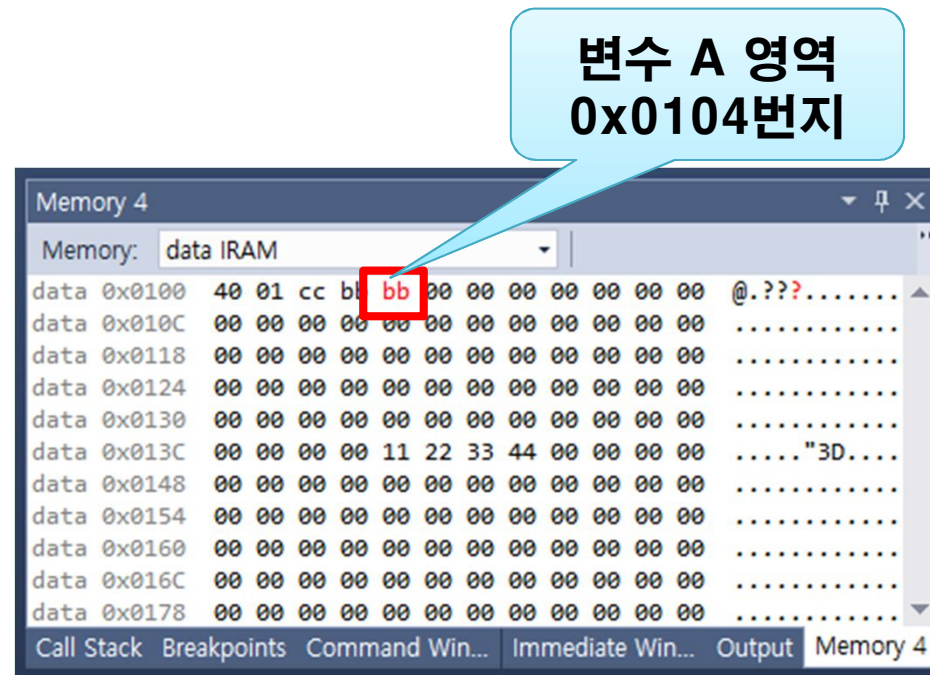
Name	Value	Type
A	0xbb	unsigned char{data}@0x0104
B	0xbb	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
ADD	0x0140	unsigned char*(data)@0x0100
* (ADD)	0x11	unsigned char{data}@0x0140

A 변수 관찰

Autos Locals Watch 1 Watch 2

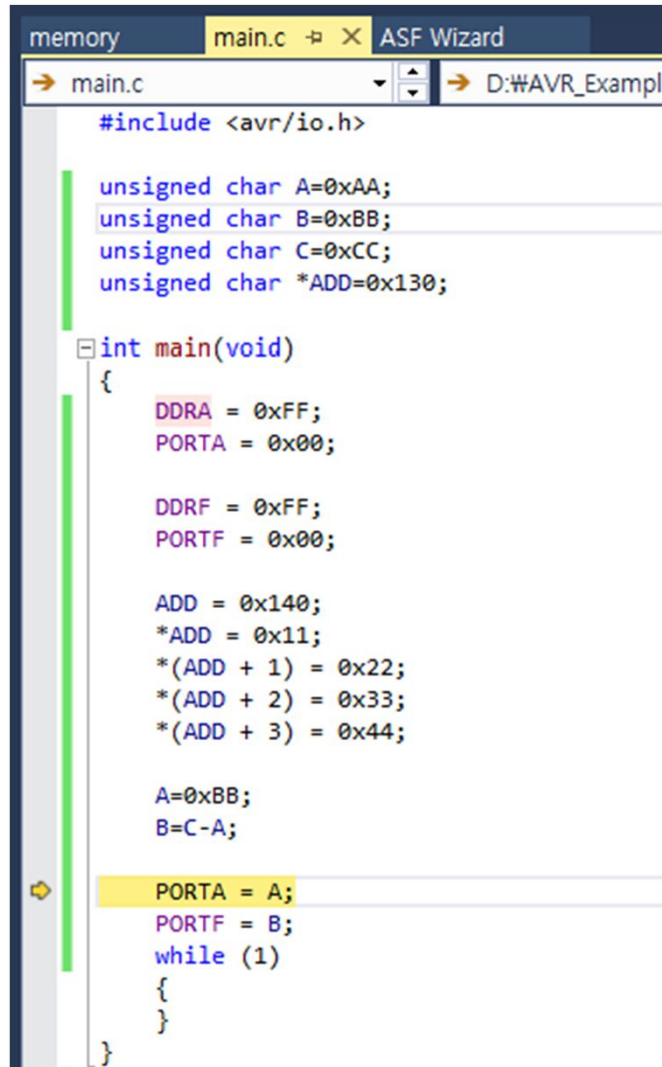
내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM 선택



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)



```
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

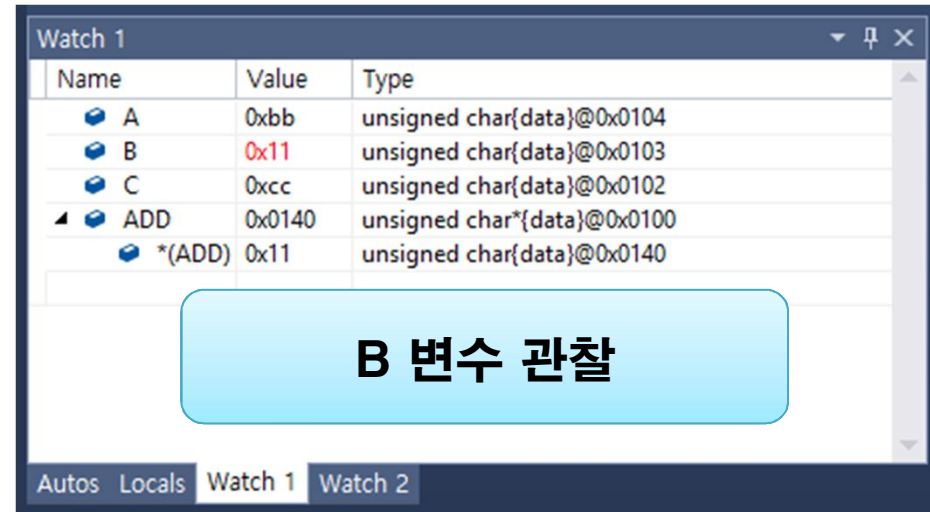
int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
```

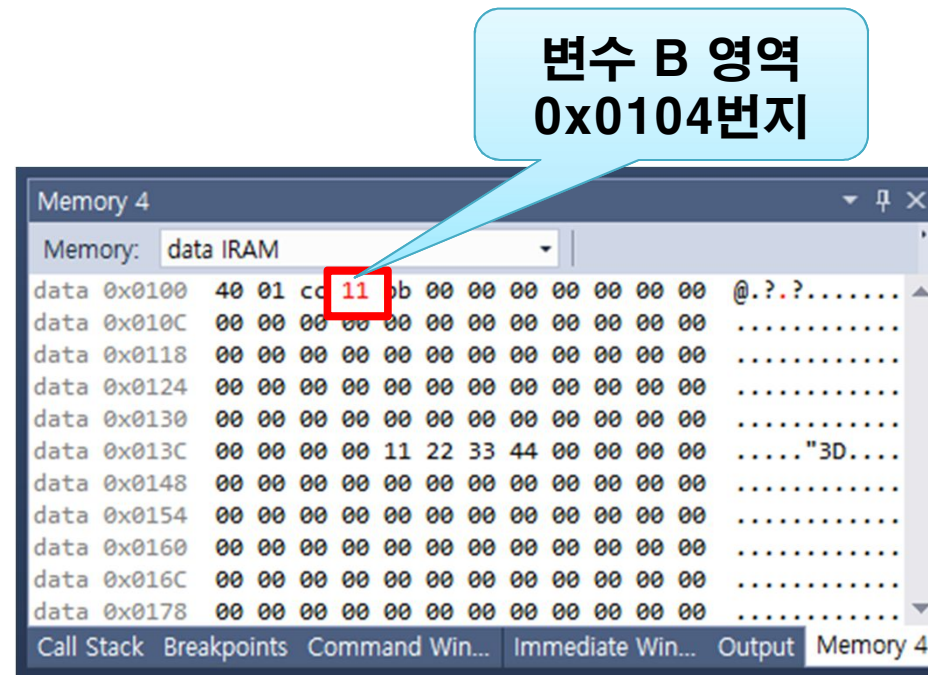


Name	Value	Type
A	0xbb	unsigned char{data}@0x0104
B	0x11	unsigned char{data}@0x0103
C	0xcc	unsigned char{data}@0x0102
ADD	0x0140	unsigned char*(data)@0x0100
*(ADD)	0x11	unsigned char{data}@0x0140

B 변수 관찰

내부 메모리와 레지스터 이해

- 내부 메모리 관찰 창
 - data IRAM 선택



내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)

```

memory    main.c  ASF Wizard
main.c
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}

```

I/O View 창에서 PORTA 레지스터 관찰

Name	Address	Value	Bits
I/O PINA	0x39	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRA	0x3A	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O PORTA	0x3B	0xBB	☒ ☒ ☐ ☐ ☒ ☒ ☒ ☒

- 내부 메모리 관찰 창
 - data MAPPED_IO 선택

Memory 4

Memory: data MAPPED_IO

data 0x0020	00	ff	00	00	00	00	00	00	00	00	00	20	
data 0x002C	00	00	00	00	fe	00	00	ff	00	00	00	00	?.....
data 0x0038	00	00	ff	bb	00	00	00	00	00	00	00	00	...?.....
data 0x0044	00	00	00	00	00	00	00	00	00	00	00	00	
data 0x0050	00	00	00	00	01	00	00	00	00	00	00	00	
data 0x005C	00	fd	10	00	00	ff	00	1f	00	00	00	00	.?.....
data 0x0068	00	00	00	00	00	00	00	00	00	f8	fe	ff	??
data 0x0074	00	00	00	00	00	00	00	00	00	00	00	00	
data 0x0080	00	00	00	00	00	00	00	00	00	00	00	00	
data 0x008C	00	00	00	00	00	00	00	00	00	06	00	00	
data 0x0098	00	00	00	20	00	06	00	00	00	00	00	00	...

Call Stack Breakpoints Command Win... Immediate Win... Output Memory 4

내부 메모리와 레지스터 이해

- 프로그램 실행 - Step Into 명령(F11)

```

memory    main.c  ASF Wizard
main.c
#include <avr/io.h>

unsigned char A=0xAA;
unsigned char B=0xBB;
unsigned char C=0xCC;
unsigned char *ADD=0x130;

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;

    DDRF = 0xFF;
    PORTF = 0x00;

    ADD = 0x140;
    *ADD = 0x11;
    *(ADD + 1) = 0x22;
    *(ADD + 2) = 0x33;
    *(ADD + 3) = 0x44;

    A=0xBB;
    B=C-A;

    PORTA = A;
    PORTF = B;
    while (1)
    {
    }
}

```

I/O View 창에서 PORTF 레지스터 관찰

Name	Address	Value	Bits
I/O PINF	0x20	0x00	☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐
I/O DDRF	0x61	0xFF	☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒
I/O PORTF	0x62	0x11	☐ ☐ ☐ ☒ ☐ ☐ ☐ ☒

- 내부 메모리 관찰 창
 - data MAPPED_IO 선택

Memory 4									
Memory: data MAPPED_IO									
data 0x0020	00	ff	00	00	00	00	00	00	20
data 0x002C	00	00	00	00	fe	00	00	ff	00
data 0x0038	00	bb	ff	bb	00	00	00	00	00
data 0x0044	00	00	00	00	00	00	00	00	00
data 0x0050	00	00	00	00	01	00	00	00	00
data 0x005C	00	fd	10	00	00	11	1f	00	00
data 0x0068	00	00	00	00	00	00	00	f8	fe
data 0x0074	00	00	00	00	00	00	00	00	00
data 0x0080	00	00	00	00	00	00	00	00	00
data 0x008C	00	00	00	00	00	00	00	06	00
data 0x0098	00	00	00	20	00	06	00	00	00